ROBUSTNESS-ORIENTED TRAINING OF LIGHTWEIGHT AUDIO DEEPFAKE DETECTORS ON AT-ADD

A Project Report Submitted
in Partial Fulfilment of the Requirements
for the Degree of

Bachelor of Technology
in Computer Science and Engineering

by

Nikkath Zabeen Shaik (Roll No. 2023BCS0060)
Bhuvan S M (Roll No. 2023BCS0198)
Abhijith Jayan (Roll No. 2023BCS0109)
Joshi Kshitij Santhosh (Roll No. 2023BCS0087)

Under the Supervision of

Dr. Sivaiah Bellamkonda



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
KOTTAYAM-686635, INDIA

October 2026

DECLARATION

We, **Nikkath Zabeen Shaik, Bhuvan S M, Abhijith Jayan, and Joshi Kshitij Santhosh** (Roll No: **2023BCS0060, 2023BCS0198, 2023BCS0109, and 2023BCS0087**), hereby declare that this report entitled "Robustness-Oriented Training of Lightweight Audio Deepfake Detectors on AT-ADD", submitted to the Indian Institute of Information Technology Kottayam in partial fulfilment of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering**, is an original work carried out by us under the supervision of **Dr. Sivaiah Bellamkonda** and has not formed the basis for the award of any degree or diploma, in this or any other institution or university. We have sincerely tried to uphold academic ethics and honesty. Whenever external information, a statement, or a result is used, it has been duly acknowledged and cited.

Kottayam-686635
October 2026

Nikkath Zabeen Shaik
Bhuvan S M
Abhijith Jayan
Joshi Kshitij Santhosh

CERTIFICATE

This is to certify that the work contained in this project report entitled "Robustness-Oriented Training of Lightweight Audio Deepfake Detectors on AT-ADD", submitted by **Nikkath Zabeen Shaik, Bhuvan S M, Abhijith Jayan, and Joshi Kshitij Santhosh** (Roll No: **2023BCS0060, 2023BCS0198, 2023BCS0109, and 2023BCS0087**) to the Indian Institute of Information Technology Kottayam in partial fulfilment of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering**, has been carried out by them under my supervision and has not been submitted elsewhere for the award of any degree.

Kottayam-686635
October 2026

Dr. Sivaiah Bellamkonda
Project Supervisor

Dr. Christina Terese Joseph
Head of the Department
Department of Computer Science and Engineering

ACKNOWLEDGMENT

We express our sincere gratitude to our project supervisor, **Dr. Sivaiah Bellamkonda**, for guidance, feedback, and support throughout this work. We thank **Dr. Christina Terese Joseph**, Head of the Department of Computer Science and Engineering, IIIT Kottayam, and the faculty and staff of the institute for providing the academic environment and resources needed to complete this project. We also thank our families and peers for their encouragement during the course of this Bachelor of Technology project.

ABSTRACT

Synthetic speech, singing, music and environmental sound can now be generated well enough to fool human listeners. A detector that is useful outside a benchmark has to cope with three things at once: it does not know the audio type in advance, the recording may have been compressed, resampled or filtered on its way to the detector, and it may have to run on a phone or another small device. Large self-supervised models score well on benchmarks but are too heavy for that last requirement, so we look at a small network whose robustness is built in during training and costs nothing at inference.

The study uses the AT-ADD Track 2 benchmark and a MobileNetV3-Small network (about 1.52 M parameters) on 80-bin log-mel input. We compare a clean-trained baseline with Frequency-Time Robustness Coherence Learning (FT-RCL), which trains the same network on pairs of clean and degraded clips with prediction and feature-signature agreement losses, and with an extension, STAR-Edge. All models were trained on a stratified subset of 13,800 training clips and evaluated on 13,287 evaluation clips drawn from all 38 generators that never appear in training.

On the official type-averaged Macro-F1, the baseline scores 0.587 (95% bootstrap interval 0.577–0.598), FT-RCL 0.589 and STAR-Edge 0.599. FT-RCL does not improve on the baseline: its pooled Macro-F1 is 0.013 lower and its EER 1.8 points higher, and the paired bootstrap intervals of both differences exclude zero. STAR-Edge gives the best AUROC (0.866 against 0.852) at an EER equal to the baseline. All three models are close to chance on music and sound. The gap between development scores (0.91–0.93) and evaluation scores (0.66–0.67) is explained by the benchmark itself: the development split shares every fake generator with the training split, whereas the evaluation split shares none.

We also audited our own pipeline. The pilot robustness test reported in an earlier version of this work used only fake music clips, and an earlier implementation of the feature-signature module constrained only three of its eight bands, so the robustness claim in the original title is not supported by the evidence and the title has been changed. No latency or quantised-accuracy measurements were made; the network needs 36.7 million multiply-accumulates per clip and 6.2 MB of weights.

Keywords: audio deepfake detection; all-type audio; AT-ADD Track 2; consistency regularisation; log-mel spectrogram; MobileNetV3-Small; channel degradation; unseen generators; shortcut learning; equal error rate.

CONTENTS

List of Figures ix

List of Tables x

List of Abbreviations xi

List of Symbols xii

1 Introduction 1

1.1 Background and motivation 1
1.2 Challenges 1
1.3 Problem statement 2
1.4 Research gap and objectives 2
1.5 Contributions 3
1.6 Organisation of the report 4

2 Literature Review 5

2.1 From speaker verification to deepfake detection 5
2.2 All-type benchmarks 5
2.3 Compact detectors and frontends 5
2.4 Robustness to compression, noise and channel change 6
2.5 Consistency regularisation 7
2.6 Shortcut learning in spoofing corpora 7
2.7 Efficient inference 7
2.8 Research gap 7

3 Proposed Methodology 8

3.1 System overview 8
3.1.1 Design principles 8
3.2 Problem formulation 8
3.3 Dataset and evaluation protocol 10
3.3.1 AT-ADD Track 2 10
3.3.2 Generator overlap between the splits 10
3.3.3 Data used in the reported experiments 10
3.4 Audio preprocessing and log-mel frontend 11
3.4.1 Loading and segmentation 11
3.4.2 Short-time Fourier transform and mel filtering 11

3.4.3 Global normalization . . . 12
3.5 Synthetic degradation (paired views) . . . 12
3.5.1 Degradation catalogue . . . 12
3.5.2 Severity levels and curriculum . . . 14
3.6 Baseline model (mode M0) . . . 15
3.6.1 Architecture . . . 15
3.6.2 Class-weighted cross-entropy loss . . . 15
3.7 FT-RCL detector . . . 15
3.7.1 Architecture and subband signatures . . . 15
3.7.2 Loss functions . . . 17
3.7.3 Total loss and training modes . . . 18
3.8 STAR-Edge extension . . . 19
3.9 Optimization and checkpoint selection . . . 20
3.9.1 Optimiser and schedule . . . 20
3.9.2 Checkpoint selection . . . 20
3.10 Summary of implementation corrections . . . 20
3.11 Training versus inference . . . 21
3.12 Evaluation metrics . . . 21
3.12.1 Metrics . . . 21

4 Experimental Setup **23**
4.1 Compute and software . . . 23
4.2 Training budget . . . 23
4.3 Hyper-parameters . . . 23
4.4 Catalogue of experiments . . . 23

5 Preliminary Studies **25**
5.1 Frontend study on ASVspoof 2019 LA . . . 25
5.2 Backbone study on AT-ADD Track 2 . . . 26
5.3 Comparison with a published MobileNetV3 system . . . 26

6 Results and Discussion **28**
6.1 Clean evaluation . . . 28
6.2 Results per audio type . . . 30
6.3 Generalisation to unseen generators . . . 31
6.4 Development versus evaluation scores . . . 32
6.5 Training behaviour . . . 32
6.6 Shortcut cues in AT-ADD . . . 33
6.7 Degraded-audio pilot . . . 34
6.8 STAR-Edge . . . 35

6.9 Model size and cost 36
6.10 Discussion 37

7 Conclusion and Future Work **38**
7.1 Conclusion 38
7.2 Limitations 39
7.3 Ethical considerations 39
7.4 Future work 40
Individual contributions 40

A Implementation Details **41**
A.1 Configuration of the reported runs 41
A.2 Training-loop pseudocode 41

References **42**

LIST OF FIGURES

3.1 Overall end-to-end workflow of the proposed methodology. Orange blocks are used only during training; the deployed detector (green) keeps the single-view baseline inference graph. 9
3.2 Audio preprocessing and log-mel feature extraction. 12
3.3 Clean and degraded versions of one evaluation clip as waveform and log-mel spectrogram. 13
3.4 Intended severity curriculum: maximum allowed severity $\rho_{\max}$ versus epoch. In the reported runs the cap did not reach the data-loading workers. 14
3.5 MobileNetV3-Small baseline on single-channel log-mel input ($\sim$1.52 M parameters). 15
3.6 FT-RCL paired clean/degraded-view training architecture. 16
3.7 Prediction coherence and feature signature coherence. 18
3.8 Training versus inference workflow. 21

6.1 Reliability curves of the three models on the evaluation subset. 29
6.2 ROC curves on the evaluation subset. 29
6.3 EER with a per-type threshold (left) and Macro-F1 with one global threshold (right), by audio type. 30
6.4 Fake recall per unseen generator at the pooled EER threshold, sorted by baseline recall. 31
6.5 Macro-F1 on the official development split (generators shared with training) and on the evaluation subset (unseen generators). 32
6.6 Validation Macro-F1 and EER and training accuracy per epoch (official development split). 33
6.7 AUROC of cue-only classifiers on the evaluation clips (all types and music). 34
6.8 Training progress of STAR-Edge. 36

LIST OF TABLES

2.1 Representative detectors. Sizes are the approximate values given in the cited papers; "–" means we could not find a figure. 6

3.1 Clips per audio type in the official lists and in the subsets used for the main comparison (real/fake in the last column). 11
3.2 Degradation families and severity levels of the training catalogue. . . . 13
3.3 Training modes and their loss components. 18

4.1 Training configuration of the main comparison. 24
4.2 Experiments reported in this work. 24

5.1 ASVspoof 2019 LA evaluation results of the frontend and head comparison (single runs). 25
5.2 Head comparison on the complete AT-ADD Track 2 evaluation list (full training data, 20 epochs, single run). 26
5.3 Published and re-implemented MobileNetV3 systems and the project baselines on the ASVspoof 2019 LA evaluation list. 27

6.1 Clean evaluation on the 13,287-clip subset (95% bootstrap intervals in brackets). The last column is the expected calibration error. 28
6.2 Paired differences from the baseline (model minus M0, 95% bootstrap intervals). For EER a negative value is an improvement. 28
6.3 Results per audio type. Columns marked "own" use the EER threshold of the type, "global" columns use the pooled EER threshold. FPR and FNR are at the global threshold. 30
6.4 AUROC on the evaluation clips of classifiers that see only one group of cues (trained on the training clips). Values below 0.5 mean that the cue works in the opposite direction on the evaluation list. 34
6.5 Clip durations in the official training and evaluation lists: median in seconds and fraction of clips shorter than four seconds (which are repeat-padded). 34
6.6 Degraded-audio pilot of the earlier version: mean ΔMacro-F1 (clean minus conditioned) on 100 fake music clips. Shown for completeness; the values are not valid evidence (Section 6.7). 35
6.7 Cost of the network. Latency, peak memory and quantised accuracy were not measured. 36
6.8 Status of the hypotheses. 37

LIST OF ABBREVIATIONS

ABR	Asymmetric bona-fide robustness
ADD	Audio deepfake detection
ALLM	Audio large language model
AT-ADD	All-type audio deepfake detection (benchmark)
AUROC	Area under the receiver operating characteristic curve
CAD	Channel auxiliary detection
CE	Cross-entropy
CNN	Convolutional neural network
ECE	Expected calibration error
EER	Equal error rate
FNR, FPR	False-negative rate, false-positive rate
FT-RCL	Frequency-time robustness coherence learning
FTSC	Feature signature coherence
MACs	Multiply-accumulate operations
M0–M3	Training modes: clean-only baseline (M0), degraded-only (M1), dual-view with prediction coherence (M2), and the full FT-RCL objective (M3); see Table 3.3
PC	Prediction coherence
ROC	Receiver operating characteristic
SNR	Signal-to-noise ratio
SSL	Self-supervised learning
STAR-Edge	Severity-trajectory and asymmetric robustness for edge
STC	Severity trajectory coherence
STFT	Short-time Fourier transform
TTM	Text-to-music
TTS	Text-to-speech

LIST OF SYMBOLS

$x,\ \tilde{x}$	Clean waveform and its degraded counterpart
y	Class label (0 = real, 1 = fake)
$f_s,\ N$	Sampling rate (24,000 Hz) and segment length in samples (96,000)
$g(\cdot\,; c)$	Waveform-domain degradation operator for condition c
$\phi(\cdot)$	Log-mel feature extractor
$\hat{\mathbf{L}}$	Normalized log-mel input to the network
$z,\ p$	Logits and class probabilities ($p = \mathrm{softmax}(z)$)
q	Fake-class probability, used as the detection score
$z_c,\ z_d,\ q_c,\ q_d$	Logits and fake-class probabilities of the clean and degraded views
$s_c,\ s_d$	Band-wise feature signatures of the clean and degraded views
K	Number of frequency subbands (8)
$\rho,\ \rho_{\max}(e)$	Degradation severity and the curriculum cap at epoch e
$\eta(\rho)$	Severity scaling factor of the M3 gate
$\alpha_t,\ \kappa$	Audio-type weight and prediction confidence of the M3 gate
$\bar{w}$	Batch-mean robustness weight
$\beta,\ \gamma_{\mathrm{pc}},\ \gamma_{\mathrm{ftsc}}$	Loss coefficients
$J,\ \lambda$	Robustness-aware selection score and its trade-off weight
ΔM	Metric drop, $M_{\mathrm{clean}} - M_{\mathrm{conditioned}}$

Chapter 1

Introduction

1.1 Background and motivation

Neural text-to-speech, singing-voice conversion, text-to-music and general audio generators have made convincing fake audio cheap to produce. Clips travel quickly through messaging applications and social media, and a listener has almost no way to check where a recording came from. Audio deepfake detection (ADD) is the task of deciding from the signal alone whether a recording is genuine or synthetic.

The first systematic work grew out of speaker-verification security, where replay, synthesis and voice-conversion attacks led to the ASVspoof challenges and to the equal error rate (EER) as the standard metric [1], [2]. Newer benchmarks cover speech, singing, music and environmental sound together, and add channel distortions that resemble real use [3]. We use Track 2 of the AT-ADD benchmark, where the detector sees only a clip and must say real or fake without being told the audio type. This is closer to deployment than speech-only testing, but it also makes it easy for a model to look good on one type and fail on the others.

Deployment is the second motivation. Audio that reaches a detector has usually been re-encoded or resampled, sometimes mixed with noise or sent through a telephone channel, and the detector may have to run on a handset with a small memory and battery. The strongest AT-ADD entries rely on self-supervised encoders with hundreds of millions of parameters [3], [4], which do not fit that setting. A small network that learns to cope with channel changes during training, and costs the same as any small network at inference, is the alternative we examine.

1.2 Challenges

Different audio types. Speech, singing, music and environmental sound differ in harmonic structure, in how they change over time and in how much noise they carry. Artefacts that expose a vocoder in speech need not appear in a music clip made by a diffusion model. Results therefore have to be reported for each type and not only pooled.

Unseen generators. A test fake may come from a generator the model has never seen [3]. If validation data share generators with the training data, validation scores

say little about this case; Section 6.4 shows how large the difference is for AT-ADD.

Channel changes. Compression, resampling, noise and band-limiting alter the spectral detail on which detectors rely. A claim of robustness needs evidence on degraded audio, preferably degraded in ways the model was not trained on.

Shortcuts. Duration, silence, loudness or bandwidth can correlate with the label in a corpus without telling anything about synthesis [5]. Whether such cues exist in AT-ADD has to be checked.

Compute limits. An edge detector is limited in parameters, arithmetic, memory and latency. A small parameter count is necessary but does not prove real-time operation; that needs measurement on the device.

1.3 Problem statement

The input is a mono waveform of four seconds at 24 kHz, and the output is a label $y \in \{0, 1\}$ (real or fake). The test set mixes four audio types and the type is hidden. We want a detector that works on all four types, that degrades gracefully when the audio is compressed, resampled, band-limited or noisy, and that is small enough for an edge device. Because the last requirement rules out a larger inference network, the robustness has to come from training.

1.4 Research gap and objectives

The literature in Chapter 2 leaves three gaps that this project touches. Compact detectors are almost always tested on speech only. Robustness is usually pursued by data augmentation, and the extra value of an explicit agreement loss between clean and degraded views has rarely been measured against an augmented baseline. Finally, edge suitability is normally argued from parameter counts instead of measured cost. We address these for one fixed backbone.

The primary objective is to find out whether coherence regularisation makes a compact detector more robust to channel degradation than augmentation alone, without hurting clean performance. We state this as four hypotheses:

H1 FT-RCL reduces the mean AUROC drop over the training degradations by at least 20% relative to a clean-trained baseline, and the bootstrap interval of the difference excludes zero.

H2 FT-RCL reduces that drop compared with the same network trained on degraded audio alone.

H3 The clean EER of FT-RCL is within one percentage point of the baseline.

H4 The advantage in H1 also holds for codecs and channels not used in training.

The secondary objectives are to set up an evaluation protocol that can tell unseen-generator performance from in-generator performance, to test whether simple acoustic cues predict the label, and to measure what the network costs to run.

How far the available evidence goes is stated in Chapter 7. In short, H3 could be tested and was not met; H1, H2 and H4 could not be tested because the degraded-audio experiments that exist are not valid, as explained in Section 6.7.

1.5 Contributions

- A comparison of a clean-trained baseline, FT-RCL and the STAR-Edge extension for one compact backbone, evaluated on audio from 38 generators unseen in training, with type-averaged metrics, a single global threshold, calibration and bootstrap intervals.
- An explanation for the large gap between development and evaluation scores: the official development split contains the same 29 fake generators as the training split.
- Evidence on shortcut cues in AT-ADD, including the native sampling rate of the corpus (16 kHz for every clip we inspected), which removes the bandwidth argument for a 24 kHz frontend.
- An audit of our own pipeline that found three problems (a signature module with empty bands, a curriculum that did not take effect with persistent data-loading workers, and a robustness pilot made only of fake music), together with the corrected implementation.
- A measured cost profile of the network: parameters, multiply-accumulates and weight size.

FT-RCL is built from known parts. Agreement between predictions for perturbed copies of an input is the basis of consistency regularisation [6], [7], and curriculum scheduling and feature-level agreement are likewise established. What we add is the adaptation to compact all-type detectors (band-wise signatures and a type-, confidence- and severity-dependent weight) and a controlled comparison against a clean-trained baseline. The M1 and M2 control runs that would separate the effect of the agreement terms from the effect of augmentation were not run, so that separation is still open.

1.6 Organisation of the report

Chapter 2 reviews related work. Chapter 3 describes the method and evaluation protocol, and Chapter 4 the data use, the training budget and the experiment catalogue. Chapter 5 reports the preliminary studies that fixed the baseline. Chapter 6 presents and discusses the main results, and Chapter 7 gives the conclusions, limitations, ethical considerations and future work.

Chapter 2

Literature Review

The review starts with speech spoofing benchmarks and moves to all-type detection, compact detectors, robustness to channel changes, consistency regularisation, shortcut learning and model compression. It ends with the gap that this project works on.

2.1 From speaker verification to deepfake detection

Work on fake audio began with attacks on speaker-verification systems. Replay, synthesis and voice conversion led to the ASVspoof challenges and to the EER as the main metric [1], [2]. ASVspoof 2019 fixed a logical-access protocol for synthetic and converted speech [2], and ASVspoof 5 later added crowdsourced recordings, adversarial attacks and codec conditions [8]. Surveys describe how the problem has spread to singing, music and environmental sound [9]–[12]. They also report that mel-spectrogram CNNs remain a common baseline [9], [13] and that the most accurate systems use self-supervised encoders, at a much higher computational cost [14]. WaveFake and FakeSound add data for neural vocoders and general audio [15], [16].

2.2 All-type benchmarks

AT-ADD evaluates speech, singing, music and environmental sound together under realistic distortions [3]. Its Track 2 Macro-F1 averages the real and fake F1 scores inside each type and then averages over the four types, so good speech performance alone is not enough. The leading systems combine self-supervised encoders [3], [4] or use audio large language models with frequency-time reasoning [17], [18]. Cross-domain and in-the-wild studies show that detectors lose a lot of accuracy when the recording conditions change [19], [20].

2.3 Compact detectors and frontends

Several small detectors exist for speech. Light CNNs work on LFCC features [21], RawNet2 works on the raw waveform [22], AASIST uses graph attention with a very small parameter count [23], and SpecRNet was designed for fast inference [24]. Pham et al. fine-tune ImageNet-pretrained vision networks, including MobileNetV3 [25], on linear-filterbank images [26]. A comparison of feature types found that the best choice

depends on the network [13]. These works use speech data and clean test audio, and they report parameter counts but not measured latency. Table 2.1 lists them next to this work.

Table 2.1. Representative detectors. Sizes are the approximate values given in the cited papers; "–" means we could not find a figure.

Method	Input	Size	Data	Robustness tested	Edge tested	Limitation
LCNN [21]	LFCC / spectrogram	–	ASVspoof 2019	No	No	Speech only
RawNet2 [22]	Raw waveform	about 25 M	ASVspoof 2019	No	No	Large, speech only
AASIST-L [23]	Raw waveform	about 85 k	ASVspoof 2019	No	Size only	Speech only
SpecRNet [24]	LFCC	about 250 k	ASVspoof 2019, WaveFake	No	Size, speed	Speech only
Pham et al. [26]	Filterbank images	1.5 M	ASVspoof 2019	No	No	Speech only
XLS-R + SLS [27]	Raw (SSL)	about 300 M	ASVspoof 2021, In-the-Wild	Partly	No	Too large
Audio LLMs [17], [18]	Waveform	billions	AT-ADD, ADD sets	Partly	No	Too large
This work	80-bin log-mel	1.52 M	AT-ADD Track 2	Training degradations (pilot invalid)	Size, MACs	One backbone, one seed

2.4 Robustness to compression, noise and channel change

Surveys agree that detectors react strongly to codecs, band-limiting, reverberation and noise [10], [11]. The usual remedy is augmentation. RawBoost applies convolutive and impulsive noise and coloured additive noise to the raw waveform and improves anti-spoofing generalisation [28], and SpecAugment masks regions of the spectrogram [29]. In the ADD 2022 challenge a fine-tuned wav2vec 2.0 system dealt with low-quality and compressed audio [30]. Datasets of perturbed voices and cross-domain collections were built to measure robustness [19], [31], and adversarial attacks show further weaknesses [32]. None of these papers asks whether an explicit agreement loss between a clean clip and its degraded copy helps beyond what augmentation already gives a small network.

2.5 Consistency regularisation

Consistency regularisation penalises differences between a model's outputs for perturbed versions of the same input. It is the basis of temporal ensembling [33], unsupervised data augmentation [6] and FixMatch [34], and the Jensen–Shannon consistency term of AugMix improves robustness to corruptions [7]. FT-RCL uses the same idea for supervised detection and differs in two ways: agreement is also required between band-wise internal features, and the weight of the constraint depends on audio type, confidence and severity. Whether these two differences matter is a question for ablation, and the ablation runs are not yet available (Chapter 6).

2.6 Shortcut learning in spoofing corpora

Müller et al. found that models trained on ASVspoof 2019 relied on the length of leading and trailing silence and not on spoofing artefacts [5]. Shortcuts of this kind give good in-corpus scores and fail when conditions change. Section 6.6 runs a similar check on AT-ADD.

2.7 Efficient inference

Smaller networks such as MobileNetV3 [25], knowledge distillation [35] and 8-bit quantisation [36] are the usual routes to edge budgets, and pretrained audio networks such as PANNs have compact variants [37]. For deepfake detection these routes are rarely evaluated with measured latency, memory or quantised accuracy.

2.8 Research gap

1. Compact detectors are almost always tested on speech. Their behaviour on music and environmental sound is largely undocumented.
2. Robustness is pursued through augmentation, and the added value of agreement losses over an augmented baseline has not been isolated.
3. Edge suitability is argued from parameter counts, and measured cost is seldom reported.
4. Validation sets that share generators with the training data hide generalisation failures.

This project looks at the second point for a fixed backbone and, because of the size of the available evidence, mainly provides a careful account of the first, third and fourth.

Chapter 3

Proposed Methodology

This chapter describes the system, the data, the degradation simulator, the models and the evaluation protocol. Where an implementation problem found during this revision changes how a result should be read, it is stated in the section it affects and summarised in Section 3.10.

3.1 System overview

The system keeps one compact backbone (MobileNetV3-Small [25]) and changes only the training objective. Waveforms are cropped, converted to normalised log-mel features and classified. During robustness training each crop is also degraded and passed through the same network, and coherence losses tie the two outputs together.

3.1.1 Design principles

- **Same inference graph.** All robustness machinery is used in training. At test time the model computes one mel spectrogram and runs one forward pass.
- **A clean-trained reference.** Mode M0 (class-weighted cross-entropy on clean input) uses the same preprocessing, data and architecture as the other modes.
- **Agreement between views.** A clean clip and a degraded copy of it carry the same label, so their outputs should be similar. Plain augmentation only shows the model degraded data; it does not ask the two outputs to agree.

3.2 Problem formulation

Let $x \in \mathbb{R}^N$ be a mono waveform with sampling rate $f_s = 24{,}000\,\text{Hz}$ and fixed length $N = 4f_s = 96{,}000$ samples (four seconds). The label $y \in \{0, 1\}$ indicates real (0) or fake (1). The classifier produces logits $z \in \mathbb{R}^2$ and probabilities $p = \text{softmax}(z)$. The **detection score** used for ROC analysis and EER is the fake-class probability $q = p_1$.

For robustness training, a condition c from a finite catalogue defines a waveform-domain operator $g(\cdot\,; c)$. The paired sample $(x, \tilde{x})$ satisfies $\tilde{x} = g(x; c)$ with the same label y and the same temporal crop. Mel extraction $\phi(\cdot)$ is applied **identically** to both waveforms:

$$\mathbf{X} = \phi(x), \qquad \tilde{\mathbf{X}} = \phi(\tilde{x}). \tag{3.1}$$

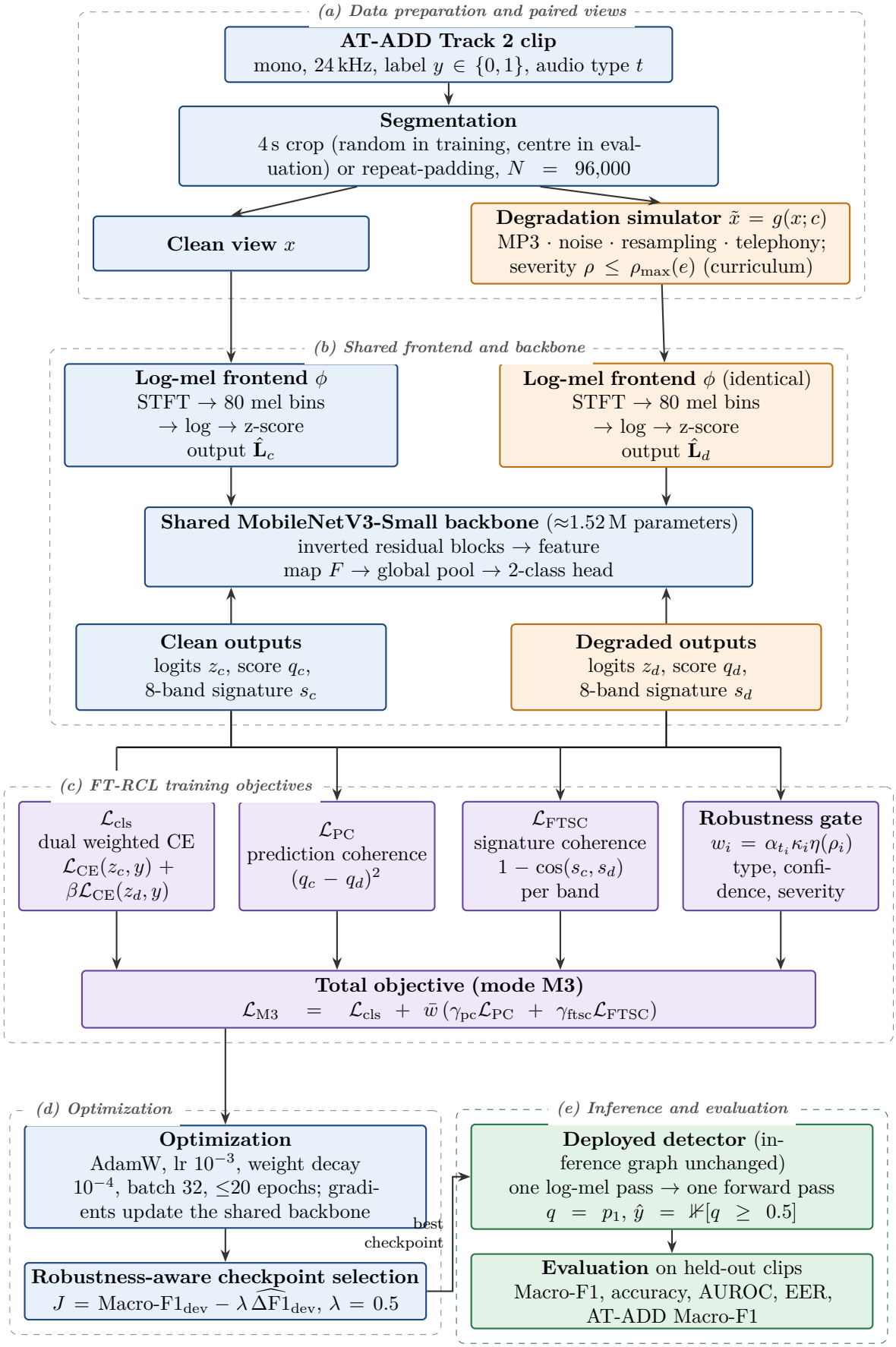


Figure 3.1. Overall end-to-end workflow of the proposed methodology. Orange blocks are used only during training; the deployed detector (green) keeps the single-view baseline inference graph.

Degradation never alters the mel hyperparameters or normalization statistics, which ensures a fair comparison with the clean baseline.

3.3 Dataset and evaluation protocol

3.3.1 AT-ADD Track 2

AT-ADD Track 2 supplies a real/fake label, the audio type (speech, singing, music, sound) and, for fake clips, the generator [3]. The training list has 46,000 clips (16,000 real, 30,000 fake), the development list 36,536 and the evaluation list 44,289, which makes 126,825 clips in total. The class ratio in training is about eight real clips to fifteen fake clips and is handled by class weighting (Section 3.6). All 5,786 evaluation clips that we inspected have a native sampling rate of 16 kHz (Section 6.6), and 24% of the training clips and 29% of the evaluation clips are shorter than four seconds and therefore need padding (Table 6.5).

3.3.2 Generator overlap between the splits

The 29 fake generators of the development list are the same 29 as in the training list. The evaluation list has 38 generators, and none of them occurs in training or development. Scores on the official development list are therefore in-generator scores, and only the evaluation list measures generalisation to unseen generators. A validation set that is disjoint in generators can be built by removing a few generators from training; this was not done for the runs reported here (Section 6.4 discusses the consequence).

3.3.3 Data used in the reported experiments

The reported baseline, FT-RCL and STAR-Edge models were trained on a label-stratified subset of 13,800 training clips (4,800 real, 9,000 fake) and validated on a subset of 10,961 development clips (1,961 real, 9,000 fake). They were evaluated on a label-stratified subset of 13,287 evaluation clips (1,961 real, 11,326 fake) that contains clips from all 38 unseen generators. No evaluation clip was used for training or model selection. Table 3.1 gives the numbers per audio type. The earlier full-data reference runs of Chapter 5 used the complete official lists.

Because 85% of the evaluation clips are fake and 61% are speech, pooled metrics mostly describe fake speech. For this reason the type-averaged Macro-F1 of the benchmark is reported next to the pooled values.

Table 3.1. Clips per audio type in the official lists and in the subsets used for the main comparison (real/fake in the last column).

	Official training	Official evaluation	Evaluation subset	Real / fake in subset
Speech	24,000	27,000	8,083	599 / 7,484
Singing	7,000	6,753	2,023	613 / 1,410
Music	8,000	4,536	1,387	160 / 1,227
Sound	7,000	6,000	1,794	589 / 1,205
Total	46,000	44,289	13,287	1,961 / 11,326

3.4 Audio preprocessing and log-mel frontend

3.4.1 Loading and segmentation

Clips are resampled to f_s if necessary. A clip longer than four seconds is cropped to length N (random start in training, centre crop in evaluation), and a shorter clip is extended by repeat-padding, i.e. the waveform is tiled instead of being followed by zeros. Repeat-padding avoids a silence cue but can create an audible discontinuity at the repeat point, which we did not examine. Music and sound clips are ten seconds long on average, so a centre crop sees less than half of them; scoring a whole recording by averaging over several windows was not tested.

3.4.2 Short-time Fourier transform and mel filtering

With a Hann window $w[n]$, frame length $L_w = 600$ samples (25 ms), hop $H = 240$ samples (10 ms), and FFT size $N_{\mathrm{FFT}} = 1024$ (the window is zero-padded to the FFT size),

$$X(m,k) = \sum_{n=0}^{L_w-1} x[n+mH]\, w[n]\, e^{-j2\pi kn/N_{\mathrm{FFT}}}, \qquad P(m,k) = |X(m,k)|^2. \tag{3.2}$$

Frames are centred using reflection padding, so a four-second segment yields $T_m = 1 + \lfloor N/H \rfloor = 401$ frames. A mel filter bank $\mathbf{W}_{\mathrm{mel}} \in \mathbb{R}^{80\times 513}$ with 80 bands (Slaney normalization) gives the mel power $\mathbf{P}_{\mathrm{mel}} = \mathbf{W}_{\mathrm{mel}}\mathbf{P}^{\top}$, and the log-mel features are

$$\mathbf{L} = \log(\mathbf{P}_{\mathrm{mel}} + \varepsilon), \qquad \varepsilon = 10^{-10}. \tag{3.3}$$

3.4.3 Global normalization

The mean μ and standard deviation σ are computed on the training split ($\mu \approx -9.19$, $\sigma \approx 6.85$ in the reference baseline) and applied in training and testing:

$$\hat{\mathbf{L}} = (\mathbf{L} - \mu)/\sigma. \tag{3.4}$$

The tensor $\hat{\mathbf{L}}$ is treated as a single-channel image of shape $1 \times 80 \times T_m$ by the CNN (Figure 3.2). Since the clips are natively 16 kHz, resampling to 24 kHz adds no information above 8 kHz, and the upper part of the mel axis carries only interpolation residue. We kept the 24 kHz setting from the preliminary studies and make no claim that it helps.



Figure 3.2. Audio preprocessing and log-mel feature extraction.

3.5 Synthetic degradation (paired views)

3.5.1 Degradation catalogue

Degradations are applied to the waveform before ϕ. Table 3.2 lists the catalogue used in training. MP3 coding uses FFmpeg (libmp3lame), additive noise is white Gaussian noise scaled to a target SNR, and the telephony condition is a fourth-order Butterworth band-pass. Each degraded clip records its family, severity and random seed.

Two properties of the catalogue need to be stated. First, the resampling condition (24 kHz $\rightarrow$ 16 kHz $\rightarrow$ 24 kHz) does nothing to this corpus, because the audio is natively 16 kHz; it is an identity operation, which explains why its measured ΔM is exactly zero (Section 6.7). Second, in the implementation used for the reported runs, the noise of a deterministic degradation was drawn from the same seed for every clip, so all clips received the same noise waveform. This has been corrected: the seed now depends on the condition and the clip identifier.

The training catalogue and the evaluation conditions are the same families. A test of generalisation therefore needs conditions that were not used in training, for instance AAC, Opus, G.711 telephony, reverberation and packet loss. Such conditions are implemented in the evaluation code but their results are not available (Section 6.7).

Table 3.2. Degradation families and severity levels of the training catalogue.

Family	Setting	Severity ρ
Resample	24 kHz $\rightarrow$ 16 kHz $\rightarrow$ 24 kHz (identity on this corpus)	1
MP3 codec	128 / 64 / 32 kbps	2 / 3 / 4
Additive noise	SNR 20 / 10 / 5 dB	2 / 3 / 4
Telephony	Band-pass 300–3400 Hz	3

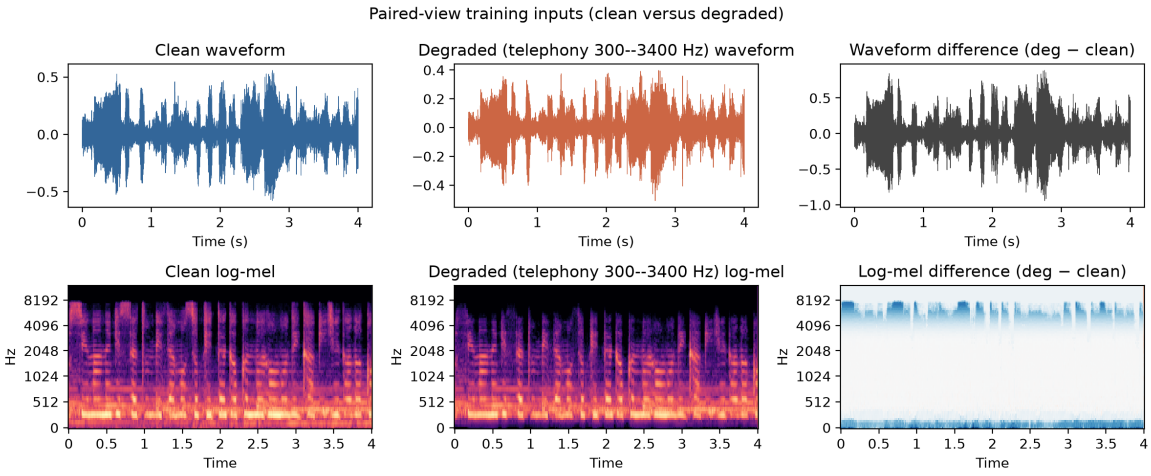


Figure 3.3. Clean and degraded versions of one evaluation clip as waveform and log-mel spectrogram.

3.5.2 Severity levels and curriculum

The severity ρ is an ordinal rank used to schedule the curriculum and to scale the gate. It increases within a family as the bitrate or SNR falls, but the ranks of different families are a design choice and not a calibrated quantity: MP3 at 128 kbps and noise at 20 dB share rank 2 without being equally damaging. The agreement between assigned rank and measured drop could not be checked because the degraded-audio results are not valid.

Let $\rho_{\max}(e)$ be the largest severity allowed at epoch e, with milestones at epochs 6 and 13:

$$\rho_{\max}(e) = \begin{cases} 1 & e < 6, \\ 3 & 6 \le e < 13, \\ 4 & e \ge 13. \end{cases} \tag{3.5}$$

Only conditions with $\rho \le \rho_{\max}(e)$ should be sampled (Figure 3.4). Two facts limit what this schedule meant in practice. The FT-RCL run stopped at epoch 12 by early stopping, so it never reached the $\rho = 4$ stage, and its best checkpoint is from epoch 7. More importantly, the training loader used persistent worker processes, each of which keeps its own copy of the dataset; the severity cap was set in the main process and never reached the workers, so every worker sampled all severities from the first epoch. The curriculum was therefore not in effect in the reported runs. The cap is now kept in shared memory and each worker has its own random stream.

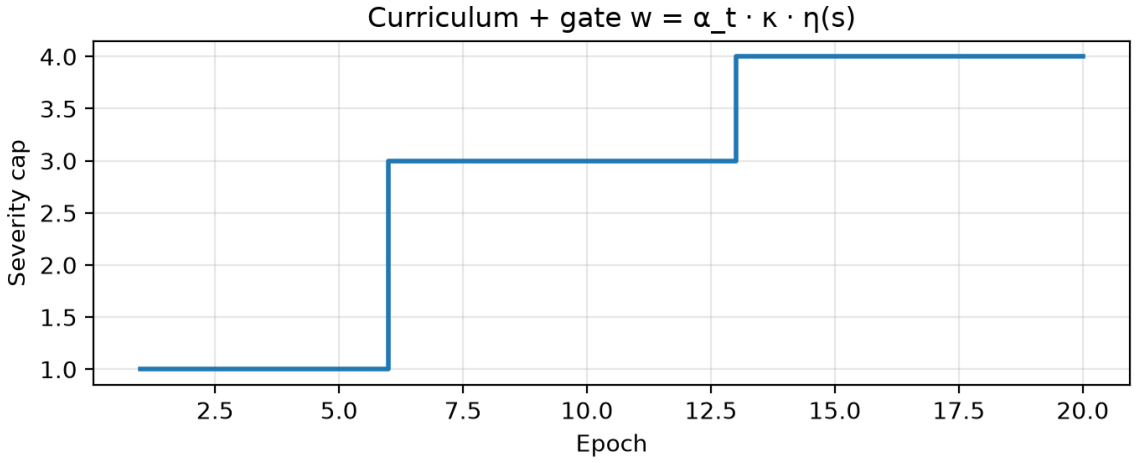


Figure 3.4. Intended severity curriculum: maximum allowed severity $\rho_{\max}$ versus epoch. In the reported runs the cap did not reach the data-loading workers.

The severity factor of the M3 gate (Section 3.7.2) is

$$\eta(\rho) = \min\left(1,\ \frac{\rho}{\rho_{\max}(e)}\right). \tag{3.6}$$

3.6 Baseline model (mode M0)

3.6.1 Architecture

The baseline uses **MobileNetV3-Small** adapted to single-channel mel input (without ImageNet pre-training) and a two-logit classifier head, giving 1,519,618 parameters. The forward pass is $\hat{\mathbf{L}} \mapsto z = f_\theta(\hat{\mathbf{L}})$ (Figure 3.5). Depthwise-separable convolutions and inverted bottlenecks keep the cost low. In the backbone comparison of Section 5.2 the CNN head gave a lower EER than a GRU or LSTM head on the same extractor, in single runs. Only variants of this one backbone were tried, and no pretrained model was used, so we do not claim that it is the best small network for the task.

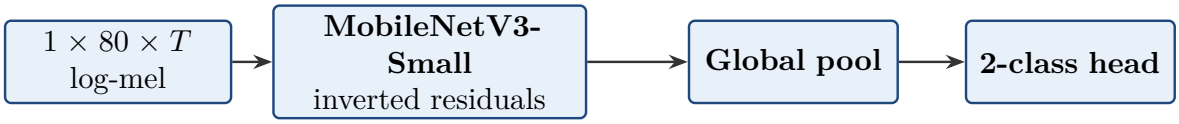


Figure 3.5. MobileNetV3-Small baseline on single-channel log-mel input (∼1.52 M parameters).

3.6.2 Class-weighted cross-entropy loss

With class counts n_0, n_1 and total $n = n_0 + n_1$, the weights $\omega_j = n/(2n_j)$ balance the real and fake classes:

$$\mathcal{L}_{\mathrm{CE}}(z, y) = -\sum_{j \in \{0,1\}} \omega_j \mathbb{1}[y = j] \log p_j. \tag{3.7}$$

For the 16:30 (real:fake) ratio of the official training list this gives $\omega_0 \approx 1.44$ and $\omega_1 \approx 0.77$. Mode **M0** (clean-only training) minimizes $\mathcal{L}_{\mathrm{CE}}$ on clean batches only.

3.7 FT-RCL detector

3.7.1 Architecture and subband signatures

The FT-RCL detector shares the MobileNetV3 backbone and also returns the last feature map $F \in \mathbb{R}^{B \times C \times H \times W}$ with $C = 576$ (Figure 3.6). The signature module splits the frequency axis into $K = 8$ bands. For band k with rows $[h_k, h_{k+1})$,

$$b_k = \mathrm{mean}_{h,w}\left(F_{:,:,h_k:h_{k+1},:}\right) \in \mathbb{R}^{B \times C}. \tag{3.8}$$

The bands are stacked into $s^{(0)} \in \mathbb{R}^{B\times K\times C}$ and normalised along the channel axis:

$$s = \text{normalize}\Big(s^{(0)},\ p = 2,\ \text{dim} = -1\Big). \tag{3.9}$$

The question of how eight bands arise from a feature map with few rows has a direct answer: they do not. The backbone reduces frequency resolution by a factor of 32, so an input with 80 mel bins gives $H = 3$ rows at the last layer. The implementation used for the reported runs assigned one row to each of the first three bands and left bands four to eight empty. The mean of an empty slice is undefined, so five of the eight signature bands were not-a-number, the logged FTSC and total loss were not-a-number in every epoch (the loss columns of the saved training logs are empty), and the training loss could not be used for monitoring. The gradients were nevertheless finite, because the undefined bands are not connected to the feature map, so training went on and the FTSC term constrained only the first three bands (one row each). We reproduced this behaviour: with the old module the loss is undefined while all parameter gradients are finite. The earlier explanation that the loss was unstable under mixed precision was wrong; the runs used a CPU and no mixed precision (Chapter 4).

The corrected module averages over time and then divides the H rows into K possibly overlapping bins with the boundaries used by adaptive average pooling, so that no band is empty. With $H = 3$ the eight bands are still an interpolated view of three rows and not eight independent frequency bands. Taking the feature map from an earlier stage with finer frequency resolution would be a better design and is left for future work. The results of Chapter 6 come from the old module, so they describe FTSC on three bands.

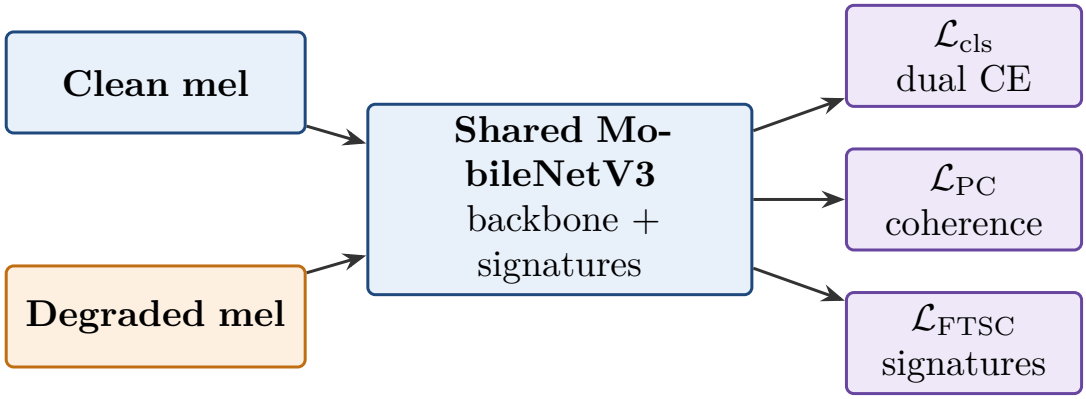


Figure 3.6. FT-RCL paired clean/degraded-view training architecture.

3.7.2 Loss functions

Let z_c, z_d be the logits from the clean and degraded mel inputs, q_c, q_d the corresponding fake-class probabilities, and s_c, s_d the signatures.

Dual classification

With the class-weighted cross-entropy of Equation (3.7),

$$\mathcal{L}_{\text{cls}} = \mathcal{L}_{\text{CE}}(z_c, y) + \beta\, \mathcal{L}_{\text{CE}}(z_d, y), \qquad \beta = 0.5. \tag{3.10}$$

Prediction coherence (PC)

$$\mathcal{L}_{\text{PC}} = \frac{1}{B} \sum_{i=1}^{B} \Big(q_{c,i} - q_{d,i}\Big)^2. \tag{3.11}$$

If, for one clip, $q_c = 0.80$ and $q_d = 0.55$, the clip adds 0.0625 to the sum and pushes the degraded prediction toward the clean one. PC cannot tell right from wrong, since two views can agree on the wrong label. A squared difference of probabilities also has a weak gradient when one of the two outputs is saturated near 0 or 1, which is exactly the case of a confident clean prediction and a collapsed degraded one. A Kullback–Leibler or Jensen–Shannon term on the output distributions does not have this property [7]; we did not compare them.

Feature signature coherence (FTSC)

For band k and sample i, let $s_{c,i,k}, s_{d,i,k} \in \mathbb{R}^C$ be the L2-normalized band vectors and $\cos_{i,k} = s_{c,i,k}^{\top} s_{d,i,k}$. FTSC compares the clean and degraded signatures by the cosine similarity along the channel dimension for each band and averages over bands and batch:

$$\mathcal{L}_{\text{FTSC}} = \frac{1}{B} \sum_{i=1}^{B} \frac{1}{K} \sum_{k=1}^{K} (1 - \cos_{i,k}). \tag{3.12}$$

Figure 3.7 illustrates PC and FTSC.

Type, confidence, and severity robustness gate

For sample i with audio type t_i, the gate combines a type weight α_{t_i} (defaults: speech and singing 1.0, music 0.3, sound 0.5), a confidence $\kappa_i = 2|q_{c,i} - 0.5|$ (treated as a constant, so no gradient flows through κ), and the severity factor $\eta(\rho_i)$ of Equation (3.6):

$$w_i = \alpha_{t_i}\, \kappa_i\, \eta(\rho_i), \qquad \bar{w} = \frac{1}{B} \sum_{i=1}^{B} w_i. \tag{3.13}$$

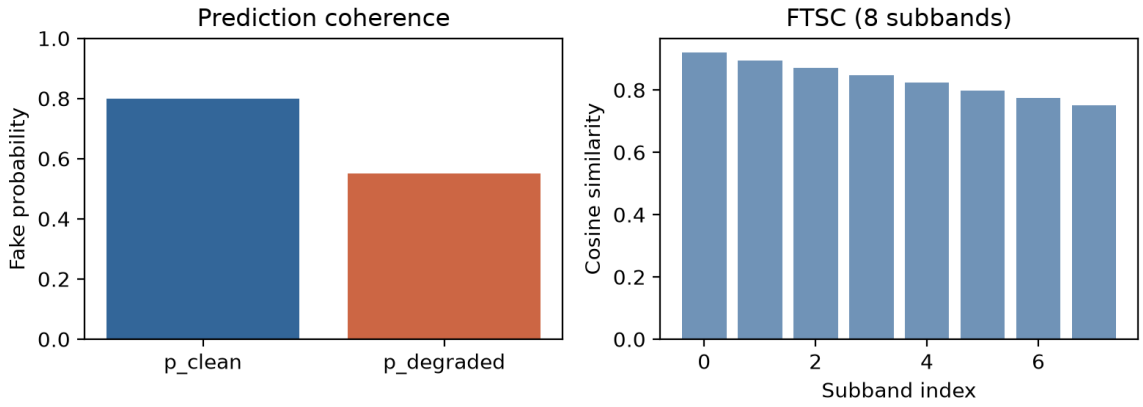


Figure 3.7. Prediction coherence and feature signature coherence.

The type weights were set by hand to reduce the constraint on the two types where the baseline is weakest; they were not estimated from data and no other values were tried. The confidence term switches the coherence loss off for uncertain clean predictions, which are the samples that probably need it most. Because the batch mean $\bar{w}$ multiplies the batch-level coherence loss, the gate scales the loss of the whole batch and does not weight individual samples. An ungated variant would show whether the gate matters; it was not run.

3.7.3 Total loss and training modes

The total loss of the full FT-RCL objective (mode M3) is

$$\mathcal{L}_{\mathrm{M3}} = \mathcal{L}_{\mathrm{cls}} + \bar{w}\left(\gamma_{\mathrm{pc}}\mathcal{L}_{\mathrm{PC}} + \gamma_{\mathrm{ftsc}}\mathcal{L}_{\mathrm{FTSC}}\right), \qquad \gamma_{\mathrm{pc}} = 1.0,\ \gamma_{\mathrm{ftsc}} = 0.5. \tag{3.14}$$

Table 3.3 summarizes how the four training modes combine the terms of Equations (3.10)–(3.12) and the gate of Equation (3.13). Mode M1 uses $\mathcal{L}_{\mathrm{CE}}(z_d, y)$ only; mode M2 uses $\mathcal{L}_{\mathrm{cls}} + \bar{w}\,\gamma_{\mathrm{pc}}\mathcal{L}_{\mathrm{PC}}$ with $\eta \equiv 1$ and $\gamma_{\mathrm{ftsc}} = 0$.

Table 3.3. Training modes and their loss components.

Mode	Network	Views used in the loss	Loss
M0	MobileNetV3-Small	Clean	$\mathcal{L}_{\mathrm{CE}}(z_c, y)$
M1	FT-RCL detector	Degraded	$\mathcal{L}_{\mathrm{CE}}(z_d, y)$
M2	FT-RCL detector	Clean and degraded	$\mathcal{L}_{\mathrm{cls}} + \bar{w}\,\gamma_{\mathrm{pc}}\mathcal{L}_{\mathrm{PC}}$ $(\eta \equiv 1)$
M3	FT-RCL detector	Clean and degraded	$\mathcal{L}_{\mathrm{cls}} + \bar{w}\,(\gamma_{\mathrm{pc}}\mathcal{L}_{\mathrm{PC}} + \gamma_{\mathrm{ftsc}}\mathcal{L}_{\mathrm{FTSC}})$

3.8 STAR-Edge extension

STAR-Edge (Severity-Trajectory and Asymmetric Robustness for Edge) adds three terms to FT-RCL. It uses the same backbone, frontend, degradation catalogue and optimisation settings.

Asymmetric bona-fide robustness (ABR). Genuine recordings suffer most when compression or noise creates artefacts that look like synthesis. For the real clips of a batch ($\mathcal{R}$), ABR penalises degraded views that look more fake than the clean view by more than a margin $m = 0.05$:

$$\mathcal{L}_{\mathrm{ABR}} = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} \max\big(0,\ q_{d,i} - q_{c,i} + m\big). \tag{3.15}$$

Severity trajectory coherence (STC). STC adds the signature and probability agreement between consecutive points of a trajectory $(s^{(0)}, \ldots, s^{(T)})$:

$$\mathcal{L}_{\mathrm{STC}} = \sum_{t=0}^{T-1} \Big[\tfrac{1}{B} \sum_i \big(1 - s_i^{(t)\top} s_i^{(t+1)}\big) + \tfrac{1}{B} \sum_i \big(q_i^{(t)} - q_i^{(t+1)}\big)^2 \Big]. \tag{3.16}$$

In the implementation the trajectory has only two points, the clean and the degraded view, so STC reduces to a second copy of the pairwise terms of FT-RCL rather than a coherence across several severities. It is applied on a random half of the training steps.

Channel auxiliary detection (CAD). Four waveform statistics (RMS, crest factor, spectral flatness, zero-crossing rate) are mapped by a small MLP to an eight-dimensional channel code u. The pooled embedding z is projected orthogonally to u, $z' = z - (z^\top \hat{u})\hat{u}$ with $\hat{u} = u/\|u\|$, and a linear head predicts the degradation family of the degraded clip from u with a cross-entropy loss. The code is detached from the projection during the first three epochs.

With weights $\rho_{\mathrm{ABR}} = 0.2$ and $\delta_{\mathrm{STC}} = 0.3$, the objective is $\mathcal{L}_{\mathrm{M3}} + 0.2\,\mathcal{L}_{\mathrm{ABR}} + \mathcal{L}_{\mathrm{CAD}} + 0.3\,\mathcal{L}_{\mathrm{STC}}$. Like the other additions, these terms are only used in training. The signature problem described above applies here as well and is worse: with CAD the signature is computed from the projected pooled embedding, which has a single row, so only one of the eight bands was defined.

3.9 Optimization and checkpoint selection

3.9.1 Optimiser and schedule

Training uses AdamW [38] with learning rate 10^{-3} and weight decay 10^{-4}, batch size 32, at most 20 epochs and early stopping with patience 5 on the selection score. The learning rate follows ReduceLROnPlateau on development Macro-F1 (factor 0.5, patience 2). The runs used a CPU, so no mixed precision was involved.

3.9.2 Checkpoint selection

The baseline keeps the epoch with the highest development Macro-F1. FT-RCL and STAR-Edge keep the epoch with the highest robustness-aware score

$$J = \mathrm{MacroF1}_{\mathrm{dev,clean}} - \lambda\, \widehat{\Delta \mathrm{F1}}_{\mathrm{dev}}, \qquad \lambda = 0.5, \tag{3.17}$$

where $\widehat{\Delta \mathrm{F1}}_{\mathrm{dev}}$ is the drop in Macro-F1 under two quick degradations. This makes the comparison unfair in a way that cannot be separated from the method: the models are chosen by different rules. The estimate itself was also unreliable. It used the first five batches of the development list, and the list is sorted by type and label, so these 160 clips were all fake music. Macro-F1 on a single class carries no information about robustness, and the logged $\widehat{\Delta \mathrm{F1}}$ values lie between -0.03 and 0.04 without a trend. In addition the estimate can be negative, in which case J rewards a model for scoring better on degraded than on clean audio. The corrected implementation uses a fixed development subset stratified by type and label, all degradation families and a drop clamped at zero, and the same selection rule is available for every model.

3.10 Summary of implementation corrections

Reviewing the code against the results exposed the problems below. The reported results of Chapter 6 come from the implementation before the corrections, except where stated; the corrections are in the code base.

1. The signature module produced empty bands, so the FTSC term was defined on three bands (one for STAR-Edge) and the logged loss was undefined.
2. The severity curriculum did not reach the persistent data-loading workers.
3. The estimate of the robustness drop used for checkpoint selection was computed on 160 fake music clips.
4. Deterministic degradations used one noise seed for all clips, and a missing codec tool silently returned the clean waveform instead of failing.

5. Checkpoints were loaded in a mode that ignores missing or unexpected weights.
6. The robustness pilot used the first rows of a sorted list (Section 6.7).

3.11 Training versus inference

Training (M3) requires paired degradations, two mel computations, dual logits and signatures, and the full loss $\mathcal{L}_{\text{M3}}$. **Inference** applies ϕ once and performs a single forward pass; signatures and degradations are omitted. This preserves the latency expected of an edge-compatible deployment (Figure 3.8).

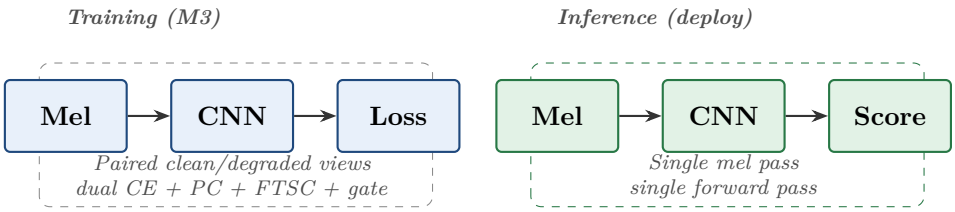


Figure 3.8. Training versus inference workflow.

3.12 Evaluation metrics

3.12.1 Metrics

For a clip with label $y \in \{0, 1\}$, the model produces logits z and probabilities $p = \text{softmax}(z)$ (Section 3.2). The **detection score** is

$$q = p_1 = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} \in [0, 1], \tag{3.18}$$

the predicted probability that the clip is fake. For a decision threshold τ, the predicted label is $\hat{y}_\tau = \mathbb{1}[q \geq \tau]$. Unless stated otherwise, $\tau = 0.5$. Let TP, FP, TN, and FN denote the counts of true positives, false positives, true negatives, and false negatives, with the fake class ($y = 1$) as the positive class. The metrics are defined as follows.

Accuracy: the fraction of correct decisions at threshold 0.5 on q,

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \tag{3.19}$$

Macro-F1: the unweighted mean of the F1 scores of the real and fake classes. For class $c \in \{0, 1\}$, with precision P_c and recall R_c,

$$\text{F1}_c = \frac{2\,P_c R_c}{P_c + R_c}, \qquad \text{Macro-F1} = \tfrac{1}{2}\left(\text{F1}_0 + \text{F1}_1\right), \tag{3.20}$$

where $P_1 = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, $R_1 = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, $P_0 = \mathrm{TN}/(\mathrm{TN}+\mathrm{FN})$, and $R_0 = \mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$. Unless stated otherwise, the overall values are computed over all evaluation clips pooled together.

AUROC: the area under the ROC curve traced by the score q. With the false-positive rate $\mathrm{FPR}(\tau) = \mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$ and the true-positive rate $\mathrm{TPR}(\tau) = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$ at threshold τ,

$$\mathrm{AUROC} = \int_0^1 \mathrm{TPR}\Big(\mathrm{FPR}^{-1}(u)\Big)\, du = \Pr\left(q_+ > q_-\right) + \tfrac{1}{2}\Pr\left(q_+ = q_-\right), \tag{3.21}$$

where q_+ and q_- are the scores of a randomly drawn fake and real clip, respectively.

EER: the error rate at the operating point where the false-positive and false-negative rates are equal. With $\mathrm{FNR}(\tau) = \mathrm{FN}/(\mathrm{FN}+\mathrm{TP})$, it is computed at the ROC point where FPR and FNR are closest,

$$\tau^* = \arg\min_{\tau} \Big|\mathrm{FPR}(\tau) - \mathrm{FNR}(\tau)\Big|, \qquad \mathrm{EER} = \tfrac{1}{2}\Big(\mathrm{FPR}(\tau^*) + \mathrm{FNR}(\tau^*)\Big). \tag{3.22}$$

AT-ADD Macro-F1: for each audio type $t \in \mathcal{T} = \{\text{speech}, \text{singing}, \text{music}, \text{sound}\}$, the F1 scores of the real and fake classes are averaged, and the four type-wise scores are then averaged without weighting [3],

$$\mathrm{F1}_{\text{AT-ADD}} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \tfrac{1}{2}\left(\mathrm{F1}_0^{(t)} + \mathrm{F1}_1^{(t)}\right), \tag{3.23}$$

where $\mathrm{F1}_c^{(t)}$ is the F1 score of class c computed on the clips of type t only. Two further quantities are reported. Because a deployed detector has one threshold for all types, the per-type results are also given at a single global threshold, taken at the pooled EER point. Calibration is summarised by the expected calibration error over 15 equal-width bins [39]. Intervals are 95% percentile intervals of a clip-level bootstrap with 1000 resamples [40]; differences between models use the same resamples, so they are paired. The robustness measure is the drop $\Delta M = M_{\text{clean}} - M_{\text{conditioned}}$; because Macro-F1 at a fixed threshold reacts strongly to a shift in the score distribution, ΔAUROC is the more reliable of the three.

No settings were tuned on evaluation data.

Chapter 4

Experimental Setup

4.1 Compute and software

The run metadata of the main experiments records PyTorch 2.14.1 and no CUDA device, so training and evaluation ran on the CPU of a laptop-class machine, in 32-bit floating point and without mixed precision. This corrects an earlier statement that the experiments used mixed-precision training on a GPU. Audio was decoded with SoundFile and resampled with librosa; MP3 coding for the degradations uses FFmpeg. The full-data reference runs of Chapter 5 were configured for GPU feature extraction. Seeds were fixed (42) and each configuration was trained once.

4.2 Training budget

Each of the three main models was trained on the subset of 13,800 training clips described in Section 3.3, for at most 20 epochs with early stopping (patience 5). The baseline reached its best validation score at epoch 20, the last epoch allowed, so it had probably not converged. The FT-RCL run stopped after epoch 12 and its best checkpoint is from epoch 7; STAR-Edge used all 20 epochs. With one seed per configuration, differences between models include training variance that the bootstrap intervals of Chapter 6 do not capture, since those intervals describe only the sampling of evaluation clips.

4.3 Hyper-parameters

4.4 Catalogue of experiments

Every experiment mentioned in this report is listed in Table 4.2. The numbering is continuous; the version numbers used in earlier drafts were internal labels and are not used here.

Modes M1 and M2 of Table 3.3 are defined but were not run.

Table 4.1. Training configuration of the main comparison.

Parameter	Value
Backbone	MobileNetV3-Small, trained from scratch, 1,519,618 parameters
Input	4 s, 24 kHz, 80-bin log-mel (window 600, hop 240, FFT 1024)
Optimiser	AdamW, learning rate 10^{-3}, weight decay 10^{-4}
Learning-rate schedule	ReduceLROnPlateau on development Macro-F1 (factor 0.5, patience 2)
Batch size, epochs, patience, seed	32, 20, 5, 42
FT-RCL coefficients	$\beta = 0.5$, $\gamma_{\text{pc}} = 1.0$, $\gamma_{\text{ftsc}} = 0.5$, $K = 8$
Type weights α_t	speech 1.0, singing 1.0, sound 0.5, music 0.3
Curriculum	epochs $[1, 6, 13] \rightarrow$ severity cap $[1, 3, 4]$ (not in effect, Section 3.5)
Checkpoint rule	Baseline: development Macro-F1. FT-RCL, STAR-Edge: J-score

Table 4.2. Experiments reported in this work.

ID	Purpose	Data and setting
P1	CNN, zero-padded 16 kHz frontend on ASVspoof 2019 LA	Official lists, 20 epochs
P2	CNN, repeat-padded 24 kHz frontend on ASVspoof 2019 LA	Official lists, 20 epochs
P3	GRU and LSTM heads under the settings of P1 and P2	Official lists, 20 epochs
P4	Re-implementation of a published MobileNetV3 system	ASVspoof 2019 LA
A1	CNN, GRU and LSTM heads on AT-ADD Track 2	Official lists, full training data, 20 epochs
M0	Clean-trained baseline	Training subset of 13,800 clips, evaluation subset of 13,287 clips
M3	FT-RCL (full objective)	As M0
S	STAR-Edge	As M0
R	Degraded-audio pilot on the baseline and FT-RCL	Fake music clips only (not valid)
C	Shortcut-cue analysis	About 6,000 training and 6,000 evaluation clips
D	Duration and padding statistics	All 126,825 clips

Chapter 5

Preliminary Studies

Before the main comparison, a series of earlier experiments settled the frontend and the backbone. They used the complete official training lists and are reported here because they motivated later choices, not as part of the main evidence. Each configuration was run once.

5.1 Frontend study on ASVspoof 2019 LA

Two frontends were compared on the ASVspoof 2019 logical-access evaluation list (71,237 clips; 25,380 training and 24,844 development clips). Setting P1 used 16 kHz audio, a four-second zero-padded window, a 400-sample frame and a 160-sample hop. Setting P2 used 24 kHz audio, repeat-padding, a 600-sample frame and a 240-sample hop. Backbone, recipe and seed were identical. The evaluation EER was 6.67% for P1 and 12.49% for P2 (Table 5.1). In P1 the recall on attack A18 was above 98.6%, in P2 it was 0.43%.

The two settings differ in padding, sampling rate, frame length and hop at the same time, so the difference cannot be put down to padding alone. It is in line with the duration shortcut that has been reported for ASVspoof 2019 [5]: zero-padded clips let a classifier read the length of the speech. But the experiment does not isolate that effect, and the idea that P2 shows "true" discrimination is a hypothesis. ASVspoof 2019 LA is also natively 16 kHz, so the 24 kHz setting only up-samples the signal and cannot expose 8–12 kHz vocoder artefacts. We did not run the single-factor experiments (padding only, sampling rate only) that would settle the question.

Table 5.1. ASVspoof 2019 LA evaluation results of the frontend and head comparison (single runs).

Setting	Head	Accuracy	Macro-F1	AUROC	EER (%)
P1: 16 kHz, zero-pad	CNN	0.909	0.820	0.9688	6.67
P1: 16 kHz, zero-pad	CNN + GRU	0.867	0.763	0.9599	7.55
P1: 16 kHz, zero-pad	CNN + LSTM	0.865	0.761	0.9666	6.82
P2: 24 kHz, repeat-pad	CNN	0.753	0.648	0.9324	12.49
P2: 24 kHz, repeat-pad	CNN + GRU	0.692	0.597	0.9198	13.82
P2: 24 kHz, repeat-pad	CNN + LSTM	0.710	0.612	0.9310	12.59

On ASVspoof 2019 the recurrent heads are within about one EER point of the

CNN head under both settings. We chose the repeat-padded 24 kHz frontend for the AT-ADD experiments to avoid the silence cue, knowing that the corpora are natively 16 kHz and that the larger bandwidth brings no new information.

5.2 Backbone study on AT-ADD Track 2

The CNN head (global average pooling) was compared with a one-layer GRU head (1.20 M parameters) [41] and a one-layer LSTM head (1.29 M parameters) [42] with hidden size 128 on the same convolutional extractor, which retains the time axis by pooling over frequency only, in the manner of earlier convolutional-recurrent spoofing detectors [43], [44]. Table 5.2 gives the results on the complete evaluation list. The CNN head has the lowest EER, 2.2 points below the LSTM head and 2.5 points below the GRU head, and the highest accuracy.

Table 5.2. Head comparison on the complete AT-ADD Track 2 evaluation list (full training data, 20 epochs, single run).

Head	Parameters	Accuracy	Macro-F1	AUROC	EER (%)
CNN (global average pooling)	1.52 M	0.757	0.678	0.8750	19.92
CNN + GRU	1.20 M	0.699	0.630	0.8425	22.43
CNN + LSTM	1.29 M	0.696	0.626	0.8622	22.15

The CNN's validation Macro-F1 on the official development list was 0.953, against 0.678 on evaluation, a gap discussed in Section 6.4. The result shows that recurrence gave no gain in these single runs; it does not show that recurrent models are worse in general, and the explanation that they overfit non-stationary audio remains a conjecture. Only a single convolutional extractor was tried, so the CNN head cannot be called optimal either. We kept it as the backbone because it is also the cheapest. A pretrained small audio network could improve the weak music and sound results but was not tried.

5.3 Comparison with a published MobileNetV3 system

Pham et al. [26] report an EER of 48% for an ImageNet-pretrained MobileNetV3 on three-channel linear-filterbank images of two-second slices of ASVspoof 2019 audio. We re-implemented their signal processing and score aggregation and trained with our own recipe (AdamW, cosine learning-rate decay, class-weighted loss, per-bin normalisation). The utterance-level EER was 16.85% with AUROC 0.9180 (segment level: 17.01%,

0.9162). The two implementations differ in optimiser, schedule, normalisation and recipe, so the gap to the published figure cannot be assigned to one cause. Likewise, the project baseline in setting P2 (EER 12.49%) differs from this re-implementation in frontend, duration and sampling rate simultaneously, so the comparison does not show that one frontend is better. Table 5.3 lists the numbers; the published F1 values are weighted or majority-class scores and are not comparable with our unweighted Macro-F1.

Table 5.3. Published and re-implemented MobileNetV3 systems and the project baselines on the ASVspoof 2019 LA evaluation list.

System	Frontend	Accuracy (%)	F1 / Macro-F1	AUROC	EER (%)
Published [26]					
MobileNetV3 (C2)	linear filterbank, 64×64×3	59.00	0.670*	0.520	48.00
ResNet-18 (C1)	linear filterbank, 64×64×3	49.00	0.580*	0.510	47.00
ConvNeXt-Tiny (C7, 28.6 M)	linear filterbank, 64×64×3	88.00	0.900*	0.960	7.50
Best ensemble	STFT, CQT, LF, Gammatone	90.00	0.910*	0.994	3.00
Re-implementation (this work)					
C2, segment level	linear filterbank, 64×64×3	64.95	0.564	0.9162	17.01
C2, utterance level	linear filterbank, 64×64×3	64.60	0.561	0.9180	16.85
Project baselines					
P2 (CNN)	80-mel, 24 kHz, repeat-pad	75.32	0.648	0.9324	12.49
P1 (CNN)	80-mel, 16 kHz, zero-pad	90.90	0.820	0.9688	6.67

*Weighted or majority-class F1 as reported by the authors; ours is unweighted Macro-F1.

Chapter 6

Results and Discussion

All numbers in this chapter come from the saved evaluation predictions of the three trained models (baseline M0, FT-RCL M3 and STAR-Edge) on the 13,287-clip evaluation subset of Section 3.3, which contains clips from all 38 unseen generators. No model was trained for this chapter beyond the runs described in Chapter 4. Each model was trained once, so intervals describe the sampling of evaluation clips and not training variance.

6.1 Clean evaluation

Table 6.1 gives the pooled results and the official type-averaged Macro-F1. The type-averaged score is the headline figure, because the pooled Macro-F1 is dominated by fake speech (61% of the evaluation clips) and is higher than the type-averaged value for every model.

Table 6.1. Clean evaluation on the 13,287-clip subset (95% bootstrap intervals in brackets). The last column is the expected calibration error.

Model	Macro-F1 (pooled)	Type-averaged Macro-F1	AUROC	EER	ECE
Baseline (M0)	0.672 [0.663, 0.681]	0.587 [0.577, 0.598]	0.852 [0.845, 0.860]	0.216 [0.207, 0.224]	0.209
FT-RCL (M3)	0.659 [0.649, 0.668]	0.589 [0.578, 0.600]	0.843 [0.835, 0.851]	0.234 [0.225, 0.243]	0.207
STAR-Edge	0.655 [0.646, 0.664]	0.599 [0.588, 0.609]	0.866 [0.859, 0.874]	0.215 [0.206, 0.223]	0.239

For the pairwise comparison the same bootstrap resamples were used for all models (Table 6.2). FT-RCL is worse than the baseline on pooled Macro-F1, AUROC and EER, with intervals that exclude zero, and is indistinguishable from it on the type-averaged score. STAR-Edge has the same EER as the baseline, a higher AUROC and a higher type-averaged Macro-F1, and a lower pooled Macro-F1.

Table 6.2. Paired differences from the baseline (model minus M0, 95% bootstrap intervals). For EER a negative value is an improvement.

Model	Δ Macro-F1	Δ type-averaged Macro-F1	Δ AUROC	Δ EER
FT-RCL (M3)	−0.013 [−0.020, −0.006]	+0.001 [−0.007, +0.011]	−0.009 [−0.016, −0.003]	+0.018 [+0.010, +0.027]
STAR-Edge	−0.017 [−0.024, −0.010]	+0.011 [+0.003, +0.020]	+0.013 [+0.007, +0.019]	−0.001 [−0.009, +0.008]

Pooled Macro-F1 is measured at the fixed threshold 0.5 and the models differ in how their scores are placed relative to it: at the pooled EER threshold the pooled

Macro-F1 values are 0.689 (baseline), 0.670 (FT-RCL) and 0.690 (STAR-Edge). The ordering of the models therefore depends on the metric, which is another reason for reporting several.

The models are poorly calibrated. The expected calibration error is 0.21 for the baseline and FT-RCL and 0.24 for STAR-Edge (Figure 6.1); the reliability curves in Figure 6.1 lie far from the diagonal. The fake share is 65% in the training subset and 85% in the evaluation subset, which shifts the best threshold, and thresholds at 0.5 are not meaningful operating points for these models.

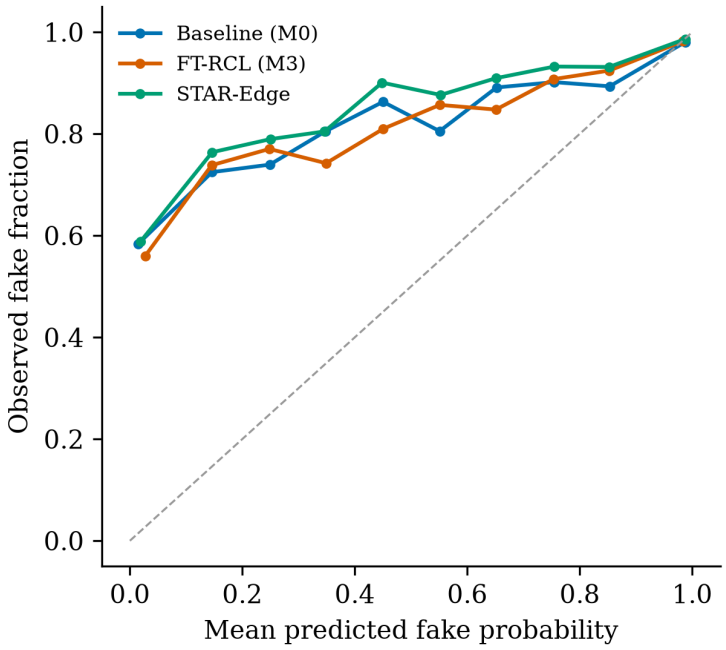
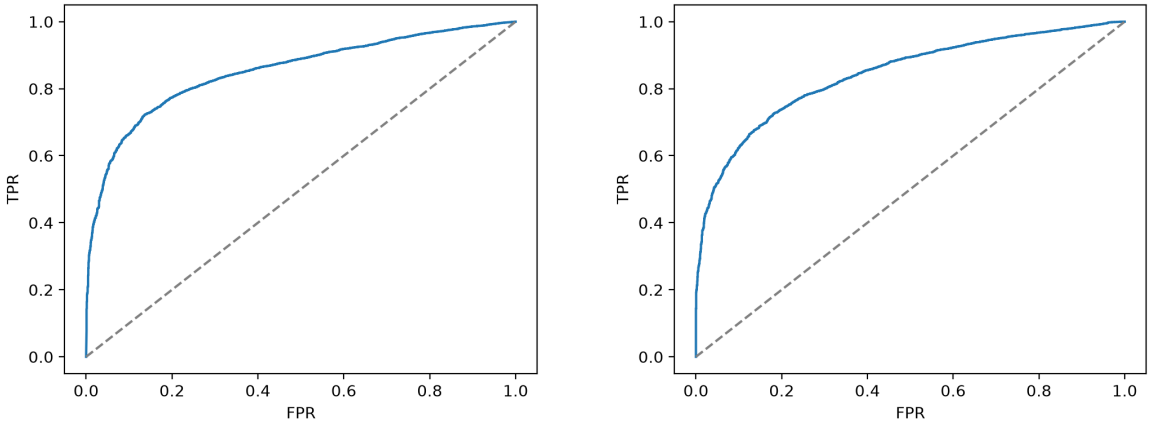


Figure 6.1. Reliability curves of the three models on the evaluation subset.



(a) Baseline (AUROC 0.852)

(b) FT-RCL (AUROC 0.843)

Figure 6.2. ROC curves on the evaluation subset.

6.2 Results per audio type

Table 6.3 gives the results per audio type, both with each type's own EER threshold and at the single pooled threshold that a type-agnostic deployment would use (Figure 6.3). Speech and singing work reasonably: the EER is between 16% and 21%, which is a high error rate for a detector and not what we would call usable. Music and sound are close to chance. The baseline's music AUROC is 0.63 and FT-RCL's is 0.55; EERs on these two types range from 0.34 to 0.47.

Table 6.3. Results per audio type. Columns marked "own" use the EER threshold of the type, "global" columns use the pooled EER threshold. FPR and FNR are at the global threshold.

Type	Model	AUROC	EER (own)	Macro-F1 (0.5)	Macro-F1 (global)	FPR	FNR
Speech	Baseline	0.909	0.162	0.695	0.717	0.282	0.095
	FT-RCL	0.905	0.179	0.665	0.682	0.254	0.129
	STAR-Edge	0.911	0.164	0.639	0.683	0.230	0.134
Singing	Baseline	0.921	0.157	0.794	0.818	0.147	0.170
	FT-RCL	0.874	0.208	0.764	0.769	0.206	0.210
	STAR-Edge	0.884	0.198	0.748	0.781	0.196	0.199
Music	Baseline	0.634	0.399	0.393	0.424	0.188	0.575
	FT-RCL	0.546	0.467	0.403	0.418	0.275	0.561
	STAR-Edge	0.669	0.383	0.449	0.489	0.250	0.455
Sound	Baseline	0.633	0.426	0.467	0.484	0.229	0.656
	FT-RCL	0.683	0.394	0.522	0.534	0.233	0.579
	STAR-Edge	0.728	0.341	0.558	0.598	0.209	0.493

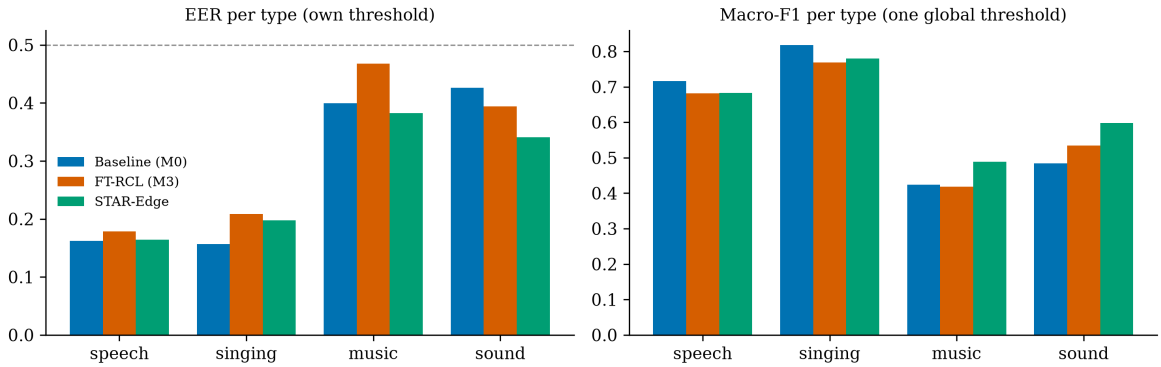


Figure 6.3. EER with a per-type threshold (left) and Macro-F1 with one global threshold (right), by audio type.

FT-RCL helps on sound and, at fixed threshold, on music, and loses on speech and singing, which is why its type-averaged score ties with the baseline. STAR-Edge is better than the baseline on music and sound and slightly worse on singing, and has

the best type-averaged score. At the global threshold, the false-negative rate on music and sound is between 45% and 66% for every model, i.e. a large share of the fake music and sound clips is accepted as real. On the evaluation subset, the music class has only 160 real clips, so music results rest on very few genuine examples.

6.3 Generalisation to unseen generators

The 38 evaluation generators differ widely in difficulty (Figure 6.4). At the pooled EER threshold the mean fake recall per generator is 0.78 for the baseline and STAR-Edge and 0.76 for FT-RCL; the number of generators with recall below 0.5 is six, five and four for baseline, FT-RCL and STAR-Edge. Speech generators such as CosyVoice 2, DiffGANTTS and VITS are detected almost perfectly. The hardest are music and sound systems: SSM-TTM is not detected at all by the baseline (recall 0.00), Auffusion reaches 0.12, MeanAudio 0.23 and ACE-Step 0.28. The speech vocoder BigVGAN is also hard (0.43 for the baseline, 0.31 for FT-RCL). A detector trained on 29 generators therefore does not extend to recent text-to-music and sound generators, which fits the weak music and sound results above.

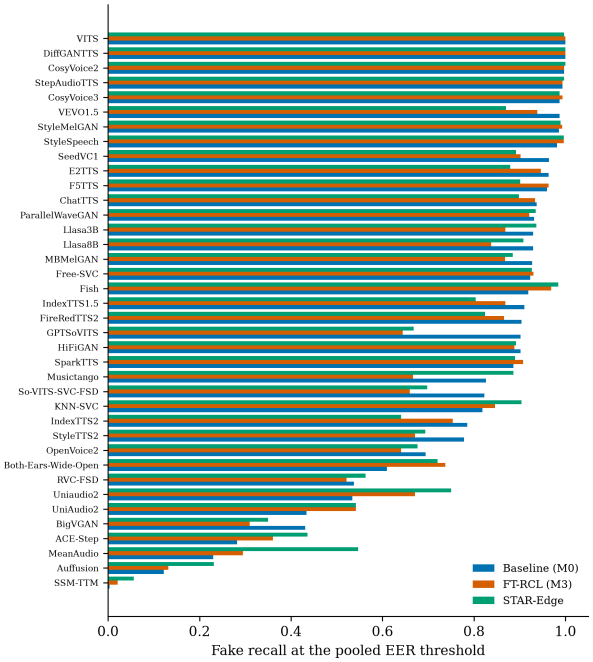


Figure 6.4. Fake recall per unseen generator at the pooled EER threshold, sorted by baseline recall.

6.4 Development versus evaluation scores

All three models score 0.91–0.93 Macro-F1 on the official development list, against 0.66–0.67 on evaluation (Figure 6.5). Overfitting to the development set is not the explanation, as the development set was only used to select an epoch. The cause is the composition of the splits (Section 3.3.2): the development list contains exactly the 29 fake generators of the training list, so its Macro-F1 measures in-generator performance, while the 38 evaluation generators are all unseen. The same gap exists for the full-data baseline (development 0.953, evaluation 0.678). A development set with held-out generators is needed for honest model selection; it can be built from the training list by removing a few generators, and would also make the comparison of checkpoint rules meaningful. We did not rerun the experiments with such a split.

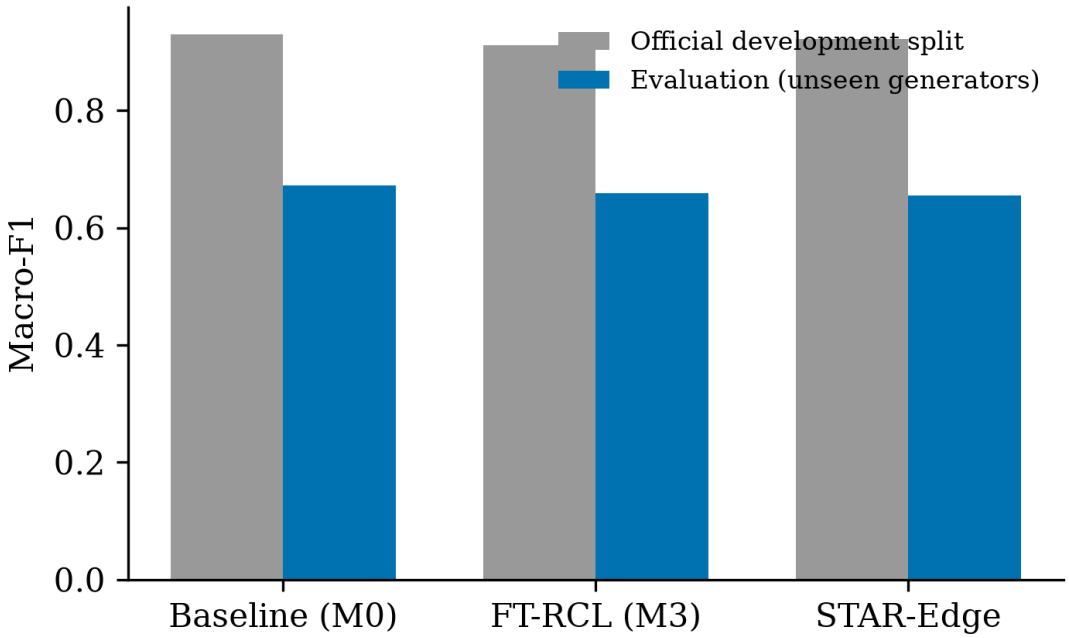


Figure 6.5. Macro-F1 on the official development split (generators shared with training) and on the evaluation subset (unseen generators).

6.5 Training behaviour

Figure 6.6 shows the validation curves. The baseline reached its best validation score at epoch 20, the last one allowed, with a training accuracy of 0.98; its development score may still have been rising and we cannot say what it would have reached with a longer schedule. FT-RCL peaked at epoch 7 and stopped at epoch 12 by early stopping, and STAR-Edge used all 20 epochs. Because the models were trained for different numbers of effective epochs and chosen by different rules (Section 3.9.2), part of the differences

in Table 6.1 may come from the protocol and not from the objectives. The training-loss column of FT-RCL and STAR-Edge is empty in the saved logs for the reason given in Section 3.7, so the loss curves of these two models cannot be shown.

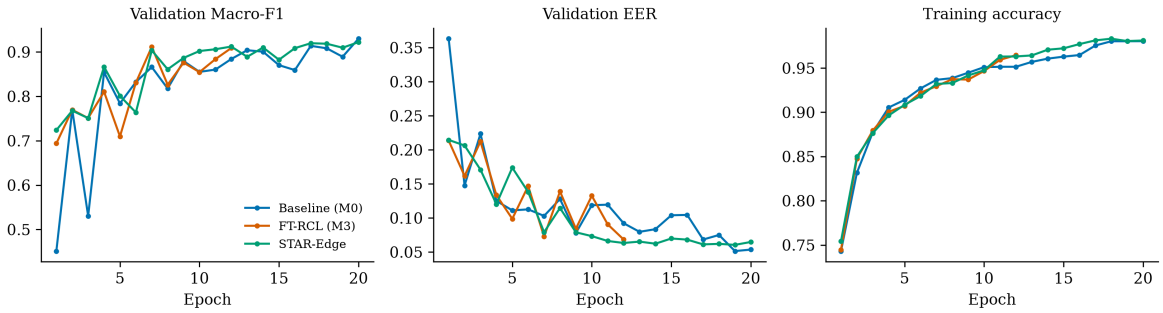


Figure 6.6. Validation Macro-F1 and EER and training accuracy per epoch (official development split).

6.6 Shortcut cues in AT-ADD

We tested whether simple cues alone predict the label. For about 6,000 training and 6,000 evaluation clips (stratified by type and label) we computed the native sampling rate, the duration, loudness (RMS and peak level), the amount of silence and the bandwidth (95% spectral roll-off and high-frequency energy ratio). A gradient-boosting classifier was fitted on each cue group on the training clips and applied to the evaluation clips.

All 5,786 sampled evaluation clips, real or fake, have a native sampling rate of 16 kHz, so the sampling rate carries no information and, as far as we inspected, the corpus has no content above 8 kHz. Loudness, silence and bandwidth cues give an AUROC between 0.49 and 0.60 on the evaluation clips in Table 6.4, which is close to chance (Table 6.4). Duration is different: it predicts the label with an AUROC well *below* 0.5 (0.36 over all types, 0.33 for music and 0.32 for sound), which means that the relation between duration and label learnt on the training list is reversed on the evaluation list. In training the fake sound clips are shorter than the real ones (mean 8.3 s against 9.9 s), whereas in evaluation the fake music clips are longer than the real ones (mean 11.3 s against 10.0 s). A classifier that learnt "shorter means fake" on the training list is therefore wrong on the evaluation list. The cues do not show a shortcut that explains the good development scores; they do show that duration differs between real and fake clips in a way that is not stable across splits, which is a risk for models that see more than four seconds.

Padding is a separate matter. Table 6.5 shows that about 40% of the speech clips are shorter than four seconds and receive repeat-padding, while music and sound clips

Table 6.4. AUROC on the evaluation clips of classifiers that see only one group of cues (trained on the training clips). Values below 0.5 mean that the cue works in the opposite direction on the evaluation list.

Cue group	All types	Speech	Music	Sound
Native sampling rate	0.500	0.500	0.500	0.500
Duration	0.362	0.539	0.331	0.323
Loudness	0.543	0.596	0.566	0.494
Silence	0.523	0.557	0.585	0.502
Bandwidth	0.512	0.519	0.517	0.519
All cues together	0.456	0.657	0.299	0.351

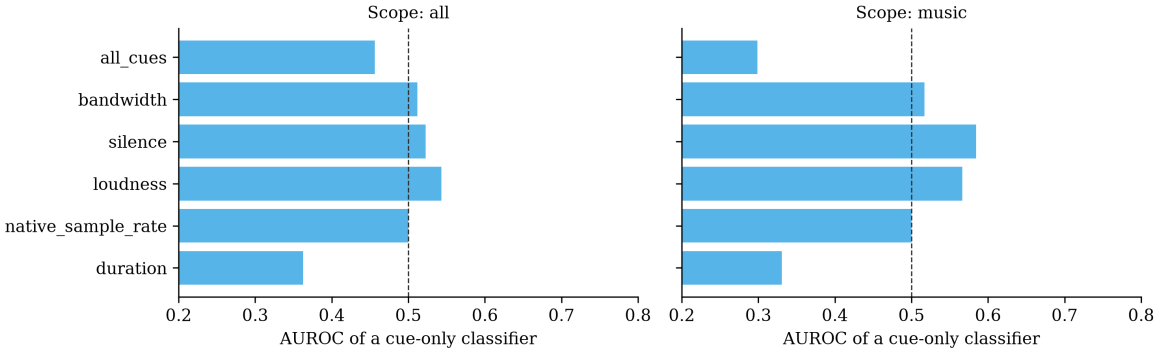


Figure 6.7. AUROC of cue-only classifiers on the evaluation clips (all types and music).

are almost all ten seconds long. The padding fraction is similar for real and fake speech, so it is not a label cue in AT-ADD, but the repeat-padded junction can be audible and we did not test its effect on detection.

Table 6.5. Clip durations in the official training and evaluation lists: median in seconds and fraction of clips shorter than four seconds (which are repeat-padded).

		Training		Evaluation	
Type	Label	Median	<4 s	Median	<4 s
Speech	real / fake	5.1 / 4.5	0.32 / 0.41	4.6 / 4.6	0.40 / 0.41
Singing	real / fake	5.1 / 5.2	0.23 / 0.23	4.7 / 5.3	0.29 / 0.24
Music	real / fake	10.0 / 10.0	0.00 / 0.00	10.0 / 10.0	0.00 / 0.00
Sound	real / fake	10.0 / 10.0	0.00 / 0.00	10.0 / 10.0	0.06 / 0.01

6.7 Degraded-audio pilot

The earlier version of this report included a conditioned evaluation on 100 clips per model, with the drop in Macro-F1 under resampling, noise and telephony (Table 6.6).

On inspection the test is not valid and no conclusion about robustness can be drawn from it, for the following reasons.

- **The clips are all fake music.** The evaluation code takes the first rows of the evaluation list, and that list is sorted by type and label. The saved output of the pilot contains only the audio type music, and Macro-F1 of a single-class sample is not defined in a useful way.
- **The zero for resampling is expected.** Every inspected clip is natively 16 kHz, so the 16 kHz round trip changes nothing.
- **The negative drops are an artefact.** A negative ΔMacro-F1 of about −0.3 under noise means that the degraded audio was scored better than the clean audio. All clips were given the same noise waveform (Section 3.5), which shifts the scores of every clip in one direction; at a fixed threshold of 0.5 and with only fake examples, such a shift raises the fraction of clips called fake and hence the Macro-F1. This is a calibration effect and not robustness.
- **No MP3 conditions.** The codec tool was not available, and the code silently returned the clean waveform for those conditions in place of failing.

Table 6.6. Degraded-audio pilot of the earlier version: mean ΔMacro-F1 (clean minus conditioned) on 100 fake music clips. Shown for completeness; the values are not valid evidence (Section 6.7).

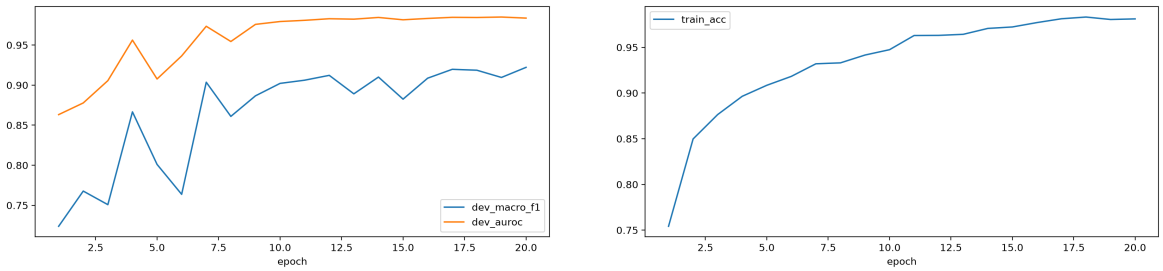
Condition	Severity	Baseline M0	FT-RCL M3
Band-limit 300–3400 Hz	3	0.115	0.134
Resample via 16 kHz	1	0.000	0.000
Noise, SNR 20 dB	2	−0.310	−0.223
Noise, SNR 10 dB	3	−0.320	−0.262
Noise, SNR 5 dB	4	−0.320	−0.268

The telephony result, in which FT-RCL loses more than the baseline (0.134 against 0.115), is subject to the same objection and should not be read as evidence for or against the method. A valid test needs a type- and label-stratified sample of the whole evaluation list, per-clip noise, AUROC-based drops, the MP3 condition and conditions that were not used in training (AAC, Opus, G.711 telephony, reverberation, packet loss). The evaluation code for this was written, but the degraded-audio results are not available, so hypotheses H1, H2 and H4 remain untested.

6.8 STAR-Edge

The STAR-Edge model was trained for the full 20 epochs and evaluated like the other two models; its results are in Tables 6.1–6.3. Relative to the baseline it gives the best

AUROC (+0.013, interval excludes zero), the best type-averaged Macro-F1 (+0.011) and the best sound and music results, at an unchanged EER and a lower pooled Macro-F1. The improvement on music and sound is the largest effect in this study. Since the signature terms in STAR-Edge were defined on one band only, and it has additional ABR and CAD terms and a different stopping point from FT-RCL, we cannot attribute the gain to any one component. Without the M1 and M2 ablations and without repeated seeds the effect should be treated as a lead to follow up and not as an established result.



(a) Development Macro-F1 and AUROC

(b) Training accuracy

Figure 6.8. Training progress of STAR-Edge.

6.9 Model size and cost

The baseline and the FT-RCL detector have the same 1,519,618 parameters, and a forward pass for one four-second clip takes 36.7 million multiply-accumulate operations, counted with a profiler on the 80×401 input (the FT-RCL detector adds only a pooling operation to this count). The saved checkpoints are 17.6 MB, but this is not the deployment size: they also contain the optimiser state. The weights alone take 6.2 MB in 32-bit floating point. Table 6.7 summarises these figures.

Table 6.7. Cost of the network. Latency, peak memory and quantised accuracy were not measured.

Model	Parameters	MACs per clip	Weights (FP32)	Saved checkpoint
Baseline (M0)	1,519,618	36.7 M	6.2 MB	17.6 MB
FT-RCL (M3)	1,519,618	36.7 M	6.2 MB	17.6 MB

Whether this satisfies an edge budget cannot be said without measuring batch-one latency including the STFT and mel step, peak memory, INT8 accuracy and, ideally, a run on a phone or single-board computer. We also have no comparison with other ways of reaching a small budget, such as a distilled or pruned larger model or a small

task-specific network like AASIST-L (about 85 k parameters [23]), so "lightweight" is judged here only against the 300 M-parameter systems of the leaderboard.

6.10 Discussion

Why FT-RCL did not help. The result is negative and we have no experiment that isolates the reason. Several explanations are consistent with the evidence. The signature loss acted on three bands, one row each, so FTSC was a weak constraint; the curriculum never operated, so there was no gradual introduction of hard degradations; the pairs were trained and chosen with a selection score computed on fake music, which makes checkpoint choice close to random among epochs 7–12; and the prediction coherence term has a weak gradient when one view is saturated. Moreover, FT-RCL differs from the baseline only through degradation-related terms while the evaluation measures clean generalisation to unseen generators, which these terms do not target. In that sense the clean evaluation is not a fair test of the idea, and the degraded-audio evaluation that would be one is missing.

Hypotheses. Table 6.8 summarises what the evidence says about the hypotheses of Chapter 1.

Table 6.8. Status of the hypotheses.

	Status	Evidence
H1	Not tested	The degraded-audio pilot is invalid (Section 6.7).
H2	Not tested	Modes M1 and M2 were not run.
H3	Not met	FT-RCL EER is 1.8 points above the baseline (interval 1.0–2.7); STAR-Edge is level with the baseline.
H4	Not tested	No valid degraded-audio results; unseen conditions not evaluated.

What the study supports. Compact CNN detectors reach an EER of about 16–21% on speech and singing from unseen generators and are close to chance on music and sound. Development scores from the official split overstate this performance by a wide margin. Under these conditions a small difference between training objectives is within what a single run can resolve for most metrics, with the exception of the AUROC advantage of STAR-Edge and the clean-EER disadvantage of FT-RCL.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

We studied a 1.52 M-parameter MobileNetV3-Small detector on AT-ADD Track 2 and compared a clean-trained baseline with two training-time robustness methods, FT-RCL and its extension STAR-Edge. The question was whether agreement between clean and degraded views makes a compact detector more robust without enlarging it. On the evidence we have, the answer is open.

What the data show is this. On 13,287 evaluation clips from 38 generators unseen in training, the baseline has a type-averaged Macro-F1 of 0.587, FT-RCL 0.589 and STAR-Edge 0.599. FT-RCL is not better than the baseline on any clean metric: its pooled Macro-F1 is 0.013 lower and its EER 1.8 points higher. STAR-Edge has the best AUROC and type-averaged score, mainly through better results on music and sound, but its pooled Macro-F1 is lower than the baseline's. All models detect speech and singing with an EER of 16–21% and are close to chance on music and sound (EER 0.34–0.47). The 0.91–0.93 Macro-F1 on the official development split is not a sign of overfitting but of a benchmark whose development fakes come from the same 29 generators as the training fakes.

What the data do not show is whether any of the methods improves robustness to compression or noise. The degraded-audio pilot of the earlier version contained only fake music, used one noise waveform for all clips, and could not apply the MP3 condition, and we withdraw its conclusions. The modes that would separate agreement losses from augmentation (M1 and M2) were not run. Latency, memory and quantised accuracy were not measured. For these reasons the title no longer claims compression robustness or edge deployment; the work is a study of robustness-oriented training of compact detectors and an audit of the pipeline that supports it.

The audit itself is a result. We found and corrected a signature module with empty bands, a curriculum that did not reach the data-loading workers, a robustness-drop estimate computed on fake music, identical noise for all clips, a silent fallback for missing codec tools and a lenient checkpoint loader. The numbers of Chapter 6 come from before these corrections and need to be reproduced with the corrected code.

7.2 Limitations

- **Training scale.** The three main models were trained on a subset of 13,800 clips, once, for at most 20 epochs, on a CPU. The baseline had not converged and the models were chosen by different rules. Absolute scores are well below those of the full-data reference run (EER 0.199).
- **Evidence on robustness.** There are no valid degraded-audio results, so H1, H2 and H4 are untested.
- **Evaluation subset.** The reported metrics use 13,287 of the 44,289 evaluation clips. The subset is stratified by label and covers all generators, but the full list would give tighter intervals.
- **Single seeds.** The bootstrap intervals describe the sampling of evaluation clips and not training variance.
- **Validation protocol.** Model selection used the official development split, which shares generators with training.
- **Edge claims.** Parameters, operations and weight size were counted; latency, memory, INT8 accuracy and device tests were not.
- **Scope of cues.** The shortcut analysis covers duration, loudness, silence and bandwidth only; speaker and recording-source leakage could not be checked with the available metadata.
- **Single backbone.** No pretrained or distilled network and no other compact architecture was tried.

7.3 Ethical considerations

A detector can be used both to protect and to harm. Published detectors help attackers tune synthetic audio until it passes, which is a reason to release code but not trained weights or tuned thresholds. A more immediate risk is a false accusation: at the error rates reported here a genuine speaker would be flagged in a noticeable share of cases, so scores from these models must not be used as the only evidence against a person. The benchmark is limited in language and speaker coverage; we did not test accents, dialects, children or people with speech impairments, and detectors tend to work worse for groups that are under-represented in training data. Running the detector on a phone keeps audio on the device, which is good for privacy, but the same capability can be used to screen calls covertly, so deployment should be transparent to the people recorded.

7.4 Future work

1. **A valid robustness evaluation.** Evaluate on a stratified sample of the whole evaluation list with per-clip noise, MP3 coding, and conditions not used in training (AAC, Opus, G.711 telephony, reverberation, packet loss), using AUROC-based drops, and check how well the severity ranks match the measured drops.
2. **Ablations and seeds.** Run M1 and M2 and an ungated variant, compare Kullback–Leibler and Jensen–Shannon agreement terms, vary the type weights and the loss coefficients, and repeat the main comparison with three to five seeds and the same checkpoint rule for every model.
3. **A generator-disjoint validation set** built from the training list, used for model selection and for training to convergence.
4. **A better signature design.** Take the signature from a layer with finer frequency resolution, and complete STAR-Edge with a trajectory over several severities.
5. **Single-factor frontend experiments** on ASVspoof (padding only, sampling rate only) and tests on additional corpora (In-the-Wild, FakeSound, ASVspoof 5).
6. **Whole-recording scoring** by averaging over windows, since music and sound clips are ten seconds long.
7. **Edge measurements and alternatives.** Measure batch-one latency with the mel step, peak memory and INT8 accuracy on a phone or single-board computer, and compare with a distilled or pruned larger model, a small task-specific detector and a pretrained compact audio network.

Individual contributions

The summary below follows the development record of the project; it should be confirmed by the team before submission.

- **Nikkath Zabeen Shaik**: FT-RCL and STAR-Edge design and implementation, training and evaluation of the main comparison, statistical analysis, report.
- **Bhuvan S M**: codec and resampling robustness pipeline and degradation experiments.
- **Abhijith Jayan**: project set-up, CNN and recurrent baselines on both corpora, re-implementation of the published comparison system.
- **Joshi Kshitij Santhosh**: to be completed by the team.

Appendix A

Implementation Details

A.1 Configuration of the reported runs

The settings of the FT-RCL run were: log-mel preprocessing with repeat-padding at 24 kHz and mel statistics computed on the official training list; batch size 32, at most 20 epochs, AdamW with learning rate 10^{-3} and weight decay 10^{-4}, early-stopping patience 5, seed 42; selection by the J-score with $\lambda = 0.5$ on five development batches; $\beta = 0.5$, $\gamma_{\mathrm{pc}} = 1.0$, $\gamma_{\mathrm{ftsc}} = 0.5$; type weights speech and singing 1.0, music 0.3, sound 0.5; curriculum epochs $[1, 6, 13]$ with severity caps $[1, 3, 4]$. In the STAR-Edge run the three STAR-Edge switches were on. The complete settings of every run are stored with its results.

A.2 Training-loop pseudocode

The loop for modes M1–M3 (Section 3.7.2), with the corrections of Section 3.10, is:

```
for epoch in 1..E:
  set severity cap from the curriculum (shared with workers)
  for clean_w, deg_w, y, type, meta in train_loader:
    if mode == M1:
      loss = CE(model(mel(deg_w)), y)
    elif mode in {M2, M3}:
      zc, sc = model(mel(clean_w))
      zd, sd = model(mel(deg_w))
      eta  = mean(severity / cap) if M3 else 1.0
      loss = dual CE + gate * (PC + FTSC)        # float32
    stop if loss is not finite
    backward(loss); AdamW step
  evaluate on the development subset; keep the best checkpoint
```

REFERENCES

[1] T. Kinnunen, M. Sahidullah, H. Kukanov, *et al.*, "The ASVspoof 2017 challenge: Assessing the limits of replay spoofing attack detection," in *Proc. Interspeech*, 2017.

[2] X. Wang, M. Todisco, M. Sahidullah, *et al.*, "ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech," *Computer Speech & Language*, vol. 64, p. 101 114, 2020. DOI: 10.1016/j.csl.2020.101114.

[3] Y. Xie, H. Cheng, J. Zhou, *et al.*, *AT-ADD: A benchmark and challenge for robust and all-type audio deepfake detection*, 2026. DOI: 10.48550/arXiv.2608.23437. arXiv: 2608.23437 [cs.SD].

[4] C. Fan, J. Cao, T. Gao, *et al.*, *Beyond speech: Dual-domain SSL fusion for unified all-type audio deepfake detection*, 2026. arXiv: 2608.29021 [cs.SD].

[5] N. M. Müller, F. Dieckmann, P. Czempin, R. Canals, K. Böttinger, and J. Williams, "Speech is silver, silence is golden: What do ASVspoof-trained models really learn?" In *Proc. ASVspoof 2021 Workshop*, 2021.

[6] Q. Xie, Z. Dai, E. Hovy, M.-T. Luong, and Q. V. Le, "Unsupervised data augmentation for consistency training," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6256–6268.

[7] D. Hendrycks, N. Mu, E. D. Cubuk, B. Zoph, J. Gilmer, and B. Lakshminarayanan, "AugMix: A simple data processing method to improve robustness and uncertainty," in *Proc. International Conference on Learning Representations*, 2020.

[8] X. Wang, H. Delgado, H. Tak, *et al.*, *ASVspoof 5: Design, collection and validation of resources for spoofing, deepfake, and adversarial attack detection using crowdsourced speech*, arXiv:2502.08857, 2025.

[9] O. A. Shaaban, R. Yildirim, and A. A. Alguttar, "Audio deepfake approaches," *IEEE Access*, vol. 11, pp. 132 652–132 682, 2023. DOI: 10.1109/ACCESS.2023.3333866.

[10] B. Zhang, H. Cui, V. Nguyen, and M. Whitty, "Audio deepfake detection: What has been achieved and what lies ahead," *Sensors*, vol. 25, no. 7, p. 1989, 2025. DOI: 10.3390/s25071989.

[11] M. Alnaqbi and R. A. Ikuesan, "A systematic review of audio deepfake detection techniques for digital investigation," *Discover Computing*, vol. 29, p. 202, 2026. DOI: 10.1007/s10791-026-10077-1.

[12] H.-H. Nguyen-Le, V.-T. Tran, D.-T. Nguyen, and N.-A. Le-Khac, "Deepfake detection across image, video, and audio: A comprehensive survey with empirical evaluation of generalization and robustness," *Artificial Intelligence Review*, vol. 59, p. 203, 2026. DOI: 10.1007/s10462-026-11608-4.

[13] M. Mcuba, A. Singh, R. A. Ikuesan, and H. S. Venter, "The effect of deep learning methods on deepfake audio detection for digital investigation," *Procedia Computer Science*, vol. 219, pp. 211–219, 2023. DOI: 10.1016/j.procs.2023.01.283.

[14] X. Li, P.-Y. Chen, *et al.*, *Where are we in audio deepfake detection? A systematic analysis over generative and detection models*, arXiv:2410.04324, 2024.

[15] J. Frank and L. Schönherr, *WaveFake: A data set to facilitate audio deepfake detection*, 2021. arXiv: 2111.02813 [cs.SD].

[16] Z. Xie, B. Li, X. Xu, Z. Liang, K. Yu, and M. Wu, *FakeSound: Deepfake general audio detection*, 2024. arXiv: 2406.08052 [cs.SD].

[17] Y. Xie, X. Guo, J. Zhou, *et al.*, *Interpretable all-type audio deepfake detection with audio LLMs via frequency–time reinforcement learning*, 2026. arXiv: 2601.02983 [cs.SD].

[18] H. Gu, J. Yi, *et al.*, "ALLM4ADD: Unlocking the capabilities of audio large language models for audio deepfake detection," in *Proc. ACM International Conference on Multimedia*, 2025. DOI: 10.1145/3746027.3755851.

[19] Y. Li *et al.*, "Cross-domain audio deepfake detection: Dataset and analysis," in *Proc. Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 4977–4983.

[20] N. M. Müller, P. Czempin, F. Dieckmann, A. Froghyar, and K. Böttinger, "Does audio deepfake detection generalize?" In *Proc. Interspeech*, 2022, pp. 2783–2787.

[21] G. Lavrentyeva, S. Novoselov, A. Tseren, M. Volkova, A. Gorlanov, and A. Kozlov, "STC antispoofing systems for the ASVspoof2019 challenge," in *Proc. Interspeech*, 2019, pp. 1033–1037.

[22] H. Tak, J. Patino, M. Todisco, A. Nautsch, N. Evans, and A. Larcher, "End-to-end anti-spoofing with RawNet2," in *Proc. IEEE ICASSP*, 2021, pp. 6369–6373.

[23] J.-w. Jung, H.-S. Heo, H. Tak, *et al.*, "AASIST: Audio anti-spoofing using integrated spectro-temporal graph attention networks," in *Proc. IEEE ICASSP*, 2022, pp. 6367–6371.

[24] P. Kawa, M. Plata, and P. Syga, "SpecRNet: Towards faster and more accessible audio deepfake detection," in *Proc. IEEE TrustCom*, 2022.

[25] A. Howard, M. Sandler, G. Chu, *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1314–1324.

[26] L. Pham, P. Lam, T. Nguyen, H. Nguyen, and A. Schindler, "Deepfake audio detection using spectrogram-based feature and ensemble of deep learning models," *arXiv preprint arXiv:2407.01777*, 2024.

[27] Q. Zhang, S. Wen, and T. Hu, "Audio deepfake detection with self-supervised XLS-R and SLS classifier," in *Proc. ACM International Conference on Multimedia*, 2024. DOI: 10.1145/3664647.3681345.

[28] H. Tak, M. Kamble, J. Patino, M. Todisco, and N. Evans, "RawBoost: A raw data boosting and augmentation method applied to automatic speaker verification anti-spoofing," in *Proc. IEEE ICASSP*, 2022, pp. 6382–6386.

[29] D. S. Park, W. Chan, Y. Zhang, *et al.*, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[30] J. M. Martín-Doñas and A. Álvarez, "The Vicomtech audio deepfake detection system based on wav2vec2 for the 2022 ADD challenge," in *Proc. IEEE ICASSP*, 2022, pp. 9241–9245.

[31] C. Gao, M. Postiglione, *et al.*, "Perturbed public voices (P2V): A dataset for robust audio deepfake detection," in *Proc. ACM Web Conference*, 2024.

[32] M. Rabhi *et al.*, "Audio-deepfake detection: Adversarial attacks and countermeasures," *Expert Systems with Applications*, vol. 250, p. 123 941, 2024. DOI: 10.1016/j.eswa.2024.123941.

[33] S. Laine and T. Aila, "Temporal ensembling for semi-supervised learning," in *Proc. International Conference on Learning Representations*, 2017.

[34] K. Sohn, D. Berthelot, N. Carlini, *et al.*, "FixMatch: Simplifying semi-supervised learning with consistency and confidence," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 596–608.

[35] G. Hinton, O. Vinyals, and J. Dean, *Distilling the knowledge in a neural network*, arXiv:1503.02531, 2015.

[36] B. Jacob, S. Kligys, B. Chen, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.

[37] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.

[38] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations*, 2019.

[39] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. International Conference on Machine Learning*, 2017, pp. 1321–1330.

[40] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[41] K. Cho, B. van Merriënboer, C. Gulcehre, *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.

[42] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[43] A. Gómez-Alanís, A. M. Peinado, J. A. González, and A. M. Gómez, "A light convolutional GRU-RNN deep feature extractor for ASV spoofing detection," in *Proc. Interspeech*, 2019, pp. 1068–1072.

[44] A. Chintha, B. Thai, S. J. Sohrawardi, *et al.*, "Recurrent convolutional structures for audio spoof and video deepfake detection," *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 5, pp. 1024–1037, 2020.